
Supervised Learning (CE)

— Unit 2 —

Supervised Learning

Supervised Learning is a type of machine learning where algorithms are trained on labeled data to make predictions or classify new data.

This approach involves using a dataset that includes both input features and their corresponding output labels.

The model learns from these labeled examples to identify patterns and relationships between inputs and outputs, enabling it to predict outputs for unseen data.

Classification problems

Predicting the category or class of an observation or data point is the goal of a classification issue in machine learning. The objective is to create a model that can learn to categorize fresh data based on correlations and patterns found in the training data.

The model is trained with labeled data, where the proper class or category of each observation is known, since classification is a supervised learning technique. The class of fresh, unknown data is then predicted using the model.

Classification issues might be binary—where there are only two classes that could exist—or multiclass—where there are multiple classes that could exist. Email spam categorization, image recognition, sentiment analysis, and disease diagnosis are a few typical instances of classification issues.

Classification Problem

There are many algorithms that can be used for classification, including logistic regression, decision trees, support vector machines (SVMs), random forests, and neural networks. The choice of algorithm depends on factors such as the size and complexity of the data, the number of classes, and the accuracy and interpretability requirements of the model.

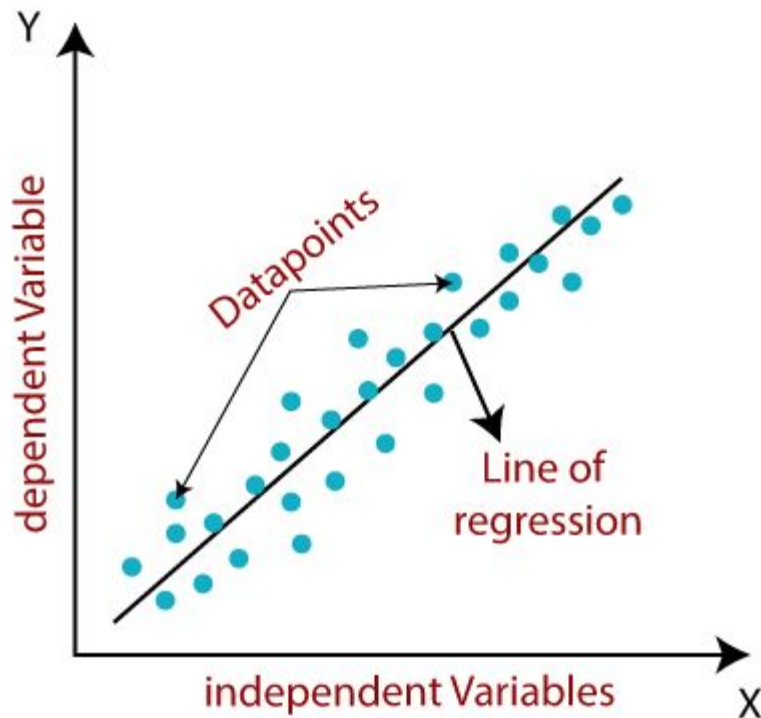
Linear Regression in Machine Learning

Linear regression is one of the easiest and most popular Machine Learning algorithms. It is a statistical method that is used for predictive analysis. Linear regression makes predictions for continuous/real or numeric variables such as **sales, salary, age, product price**, etc.

Linear regression algorithm shows a linear relationship between a dependent (y) and one or more independent (x) variables, hence called as linear regression. Since linear regression shows the linear relationship, which means it finds how the value of the dependent variable is changing according to the value of the independent variable.

Linear Regression (cont.)

The linear regression model provides a sloped straight line representing the relationship between the variables. Consider the below image:



Linear Regression (cont.)

Mathematically, we can represent linear regression as:

$$y = a_0 + a_1x + \epsilon$$

Y= Dependent Variable (Target Variable)

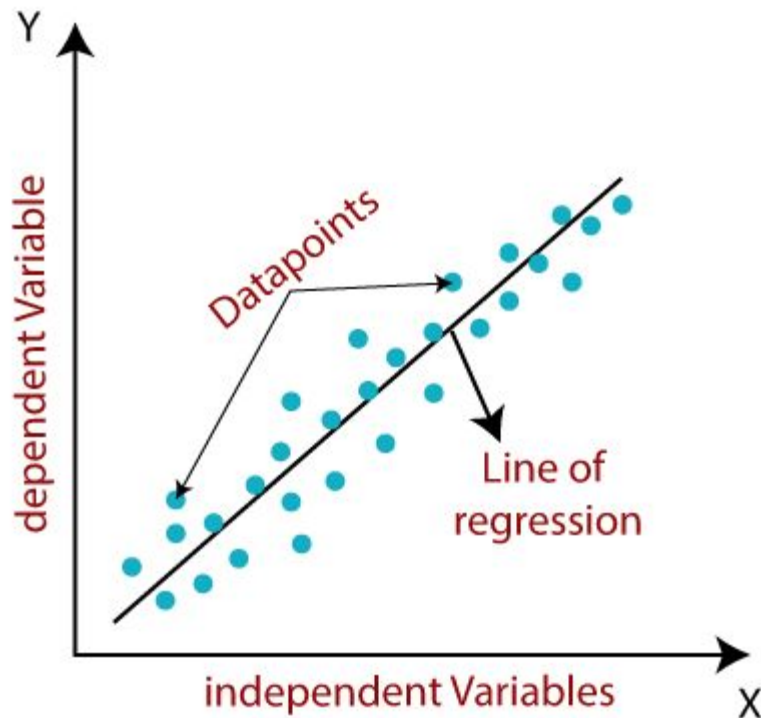
X= Independent Variable (predictor Variable)

a_0 = intercept of the line (Gives an additional degree of freedom)

a_1 = Linear regression coefficient (scale factor to each input value).

ϵ = random error

The values for x and y variables are training datasets for Linear Regression model representation.



Types of Linear Regression

Linear regression can be further divided into two types of the algorithm:

- **Simple Linear Regression:**

If a single independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Simple Linear Regression.

- **Multiple Linear regression:**

If more than one independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Multiple Linear Regression.

Finding the best fit-line

When working with linear regression, our main goal is to find the best fit line that means the error between predicted values and actual values should be minimized. The best fit line will have the least error.

The different values for weights or the coefficient of lines (a_0 , a_1) gives a different line of regression, so we need to calculate the best values for a_0 and a_1 to find the best fit line, so to calculate this we use cost function.

Cost function-

- The different values for weights or coefficient of lines (a_0, a_1) gives the different line of regression, and the cost function is used to estimate the values of the coefficient for the best fit line.
- Cost function optimizes the regression coefficients or weights. It measures how a linear regression model is performing.
- We can use the cost function to find the accuracy of the **mapping function**, which maps the input variable to the output variable. This mapping function is also known as **Hypothesis function**.

For Linear Regression, we use the **Mean Squared Error (MSE)** cost function, which is the average of squared error occurred between the predicted values and actual values. It can be written as:

$$MSE = \frac{1}{N} \sum_{i=1}^n (y_i - (a_1 x_i + a_0))^2$$

Where,

N=Total

Y_i

$(a_1 x_i + a_0)$ = Predicted value.

number

=

of

Actual

observation

value

Problem Deninition:

Find a quadratic regression model for the following data:

X	Y
1	1
2	2
3	1.3
4	3.75
5	2.25

Solution:

Let the simple linear regression model be

$$y = a + bx$$

Steps to find **a** and **b**,

First, find the mean and covariance.

Means of x and y are given by,

$$\bar{x} = \frac{1}{n} \sum x_i$$

$$\bar{y} = \frac{1}{n} \sum y_i$$

The variance of x is given by,

$$\text{Var}(x) = \frac{1}{n-1} \sum (x_i - \bar{x})^2$$

The covariance of x and y, denoted by $\text{Cov}(x, y)$ is defined as,

$$\text{Cov}(x, y) = \frac{1}{n-1} \sum (x_i - \bar{x})(y_i - \bar{y})$$

Now the values of a and b can be computed using the following formulas:

$$b = \frac{\text{Cov}(x, y)}{\text{Var}(x)}$$

$$a = \bar{y} - b\bar{x}$$

First, find the mean of x and y,

$$n = 5$$

$$\begin{aligned}\bar{x} &= \frac{1}{5}(1.0 + 2.0 + 3.0 + 4.0 + 5.0) \\ &= 3.0\end{aligned}$$

$$\begin{aligned}\bar{y} &= \frac{1}{5}(1.00 + 2.00 + 1.30 + 3.75 + 2.25) \\ &= 2.06\end{aligned}$$

Next, find the Covariance between x and y,

$$\text{Cov}(x, y) = \frac{1}{n-1} \sum (x_i - \bar{x})(y_i - \bar{y})$$

$$\begin{aligned}\text{Cov}(x, y) &= \frac{1}{4}[(1.0 - 3.0)(1.00 - 2.06) + \dots + (5.0 - 3.0)(2.25 - 2.06)] \\ &= 1.0625\end{aligned}$$

Now find the variance of x ,

$$\text{Var}(x) = \frac{1}{n-1} \sum (x_i - \bar{x}_i)^2$$

$$\begin{aligned}\text{Var}(x) &= \frac{1}{4} [(1.0 - 3.0)^2 + \dots + (5.0 - 3.0)^2] \\ &= 2.5\end{aligned}$$

Now, find the intercept and coefficients,

$$b = \frac{\text{Cov}(x, y)}{\text{Var}(x)}$$

$$a = \bar{y} - b\bar{x}$$

$$b = \frac{1.0625}{2.5}$$

$$= 0.425$$

$$a = 2.06 - 0.425 \times 3.0$$

$$= 0.785$$

Therefore, the linear regression model for the data is,
 $y = 0.785 + 0.425 x$

Multiple Linear Regression

Multiple linear regression is used to estimate the relationship between two or more independent variables and one dependent variable. You can use multiple linear regression when you want to know:

1. How strong the relationship is between two or more independent variables and one dependent variable (e.g. how rainfall, temperature, and amount of fertilizer added affect crop growth).
2. The value of the dependent variable at a certain value of the independent variables (e.g. the expected yield of a crop at certain levels of rainfall, temperature, and fertilizer addition).

Multiple linear regression formula

The formula for a multiple linear regression is:

$$y = \beta_0 + \beta_1 X_1 + \dots + \beta_n X_n + \epsilon$$

- y = the predicted value of the dependent variable
- B_0 = the y-intercept (value of y when all other parameters are set to 0)
- $B_1 X_1$ = the regression coefficient (B_1) of the first independent variable (X_1) (a.k.a. the effect that increasing the value of the independent variable has on the predicted y value)
- ... = do the same for however many independent variables you are testing
- $B_n X_n$ = the regression coefficient of the last independent variable
- ϵ = model error (a.k.a. how much variation there is in our estimate of y)

- The multiple regression of two variables x_1 and x_2 is given as follows:

$$y = f(x_1, x_2)$$

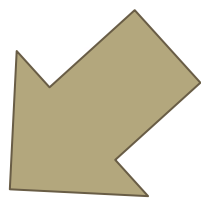
$$y = a_0 + a_1x_1 + a_2x_2$$

- In general, this is given for 'n' independent variables as:

$$y = f(x_1, x_2, \dots, x_n)$$

$$y = a_0 + a_1x_1 + a_2x_2 + \dots + a_nx_n + \varepsilon$$

- Here, $x_1, x_2, \dots, x_n$ are predictor variables, y is the dependent variable, $(a_0, a_1, a_2, \dots, a_n)$ are the coefficients of the regression equation and ε is the error term.



Data having multiple independent variables

X1 (Product 1 Sales)	X2 (Product 2 Sales)	Y (Weekly Sales)
1	4	1
2	5	6
3	8	8
4	2	12

Representing the above data in matrix form

- Here, the matrices for Y and X are given as follows:

$$X = \begin{pmatrix} 1 & 1 & 4 \\ 1 & 2 & 5 \\ 1 & 3 & 8 \\ 1 & 4 & 2 \end{pmatrix} \text{ and } Y = \begin{pmatrix} 1 \\ 6 \\ 8 \\ 12 \end{pmatrix}.$$

The coefficient of multiple linear regression equation is given as

$$a = \begin{pmatrix} a_0 \\ a_1 \\ a_2 \end{pmatrix}.$$

- The regression coefficient for multiple regression is calculated the same way as linear regression:

$$\hat{a} = ((X^T X)^{-1} X^T) Y$$

Calculation of first part

$$X^T X = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 4 & 5 & 8 & 2 \end{pmatrix} \begin{pmatrix} 1 & 1 & 4 \\ 1 & 2 & 5 \\ 1 & 3 & 8 \\ 1 & 4 & 2 \end{pmatrix} = \begin{pmatrix} 4 & 10 & 19 \\ 10 & 30 & 46 \\ 19 & 46 & 109 \end{pmatrix}$$

Calculation of the inverse


$$(X^T X)^{-1} = \begin{pmatrix} 4 & 10 & 19 \\ 10 & 30 & 46 \\ 19 & 46 & 109 \end{pmatrix}^{-1} = \begin{pmatrix} 3.15 & -0.59 & -0.30 \\ -0.59 & 0.20 & 0.016 \\ -0.30 & 0.016 & 0.054 \end{pmatrix}$$

Evaluating the second part

$$X^T X)^{-1} X^T = \begin{pmatrix} 3.15 & -0.59 & -0.30 \\ -0.59 & 0.20 & 0.016 \\ -0.30 & 0.016 & 0.054 \end{pmatrix} \times \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 4 & 5 & 8 & 2 \end{pmatrix} = \begin{pmatrix} 0.05 & 0.47 & -1.02 & 0.19 \\ -0.32 & -0.098 & 0.155 & 0.26 \\ -0.065 & 0.005 & 0.185 & -0.125 \end{pmatrix}$$

Evaluation of regression coefficient

$$\hat{a} = ((X^T X)^{-1} X^T) Y = \begin{pmatrix} 0.05 & 0.47 & -1.02 & 0.19 \\ -0.32 & -0.098 & 0.155 & 0.26 \\ -0.065 & 0.005 & 0.185 & -0.125 \end{pmatrix} \times \begin{pmatrix} 1 \\ 6 \\ 8 \\ 12 \end{pmatrix} = \begin{pmatrix} -1.69 \\ 3.48 \\ -0.05 \end{pmatrix}$$

$$a_0 = -1.69$$

$$a_1 = 3.48$$

$$a_2 = -0.05$$

Final Equation of Multiple Linear Regression

- $y = a_0 + a_1x_1 + a_2x_2$

- Hence, the constructed model is:

- $y = -1.69 + 3.48x_1 - 0.05x_2$

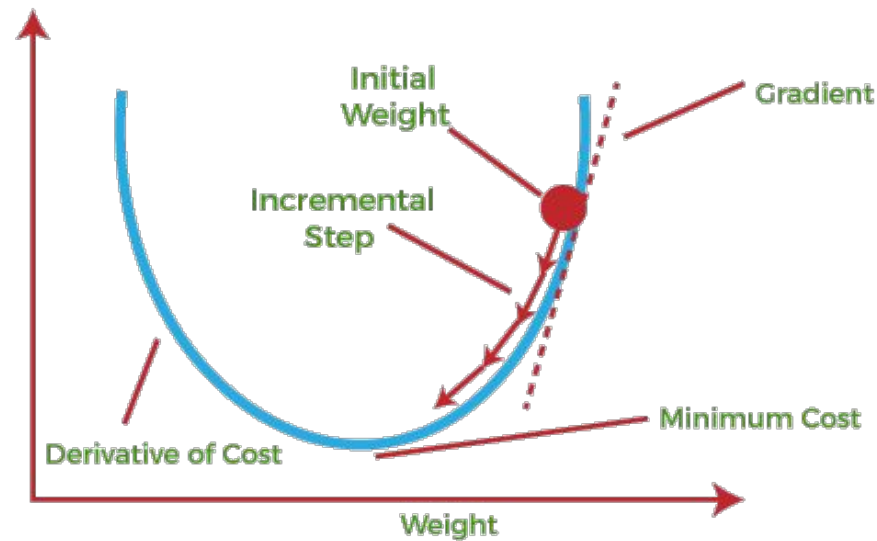
Gradient Descent

Gradient Descent is defined as one of the most commonly used iterative optimization algorithms of machine learning to train the machine learning and deep learning models.

It helps in finding the local minimum of a function.

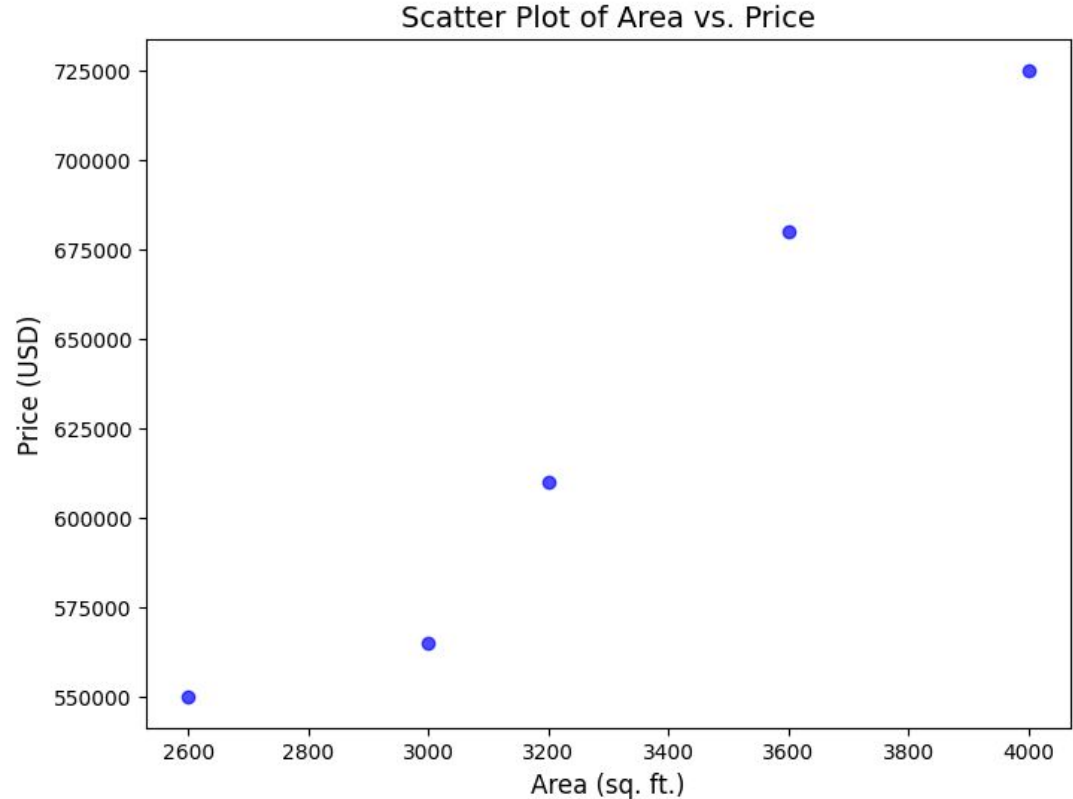
Gradient Descent

- If we move towards a negative gradient or away from the gradient of the function at the current point, it will give the **local minimum** of that function.
- Whenever we move towards a positive gradient or towards the gradient of the function at the current point, we will get the **local maximum** of that function.

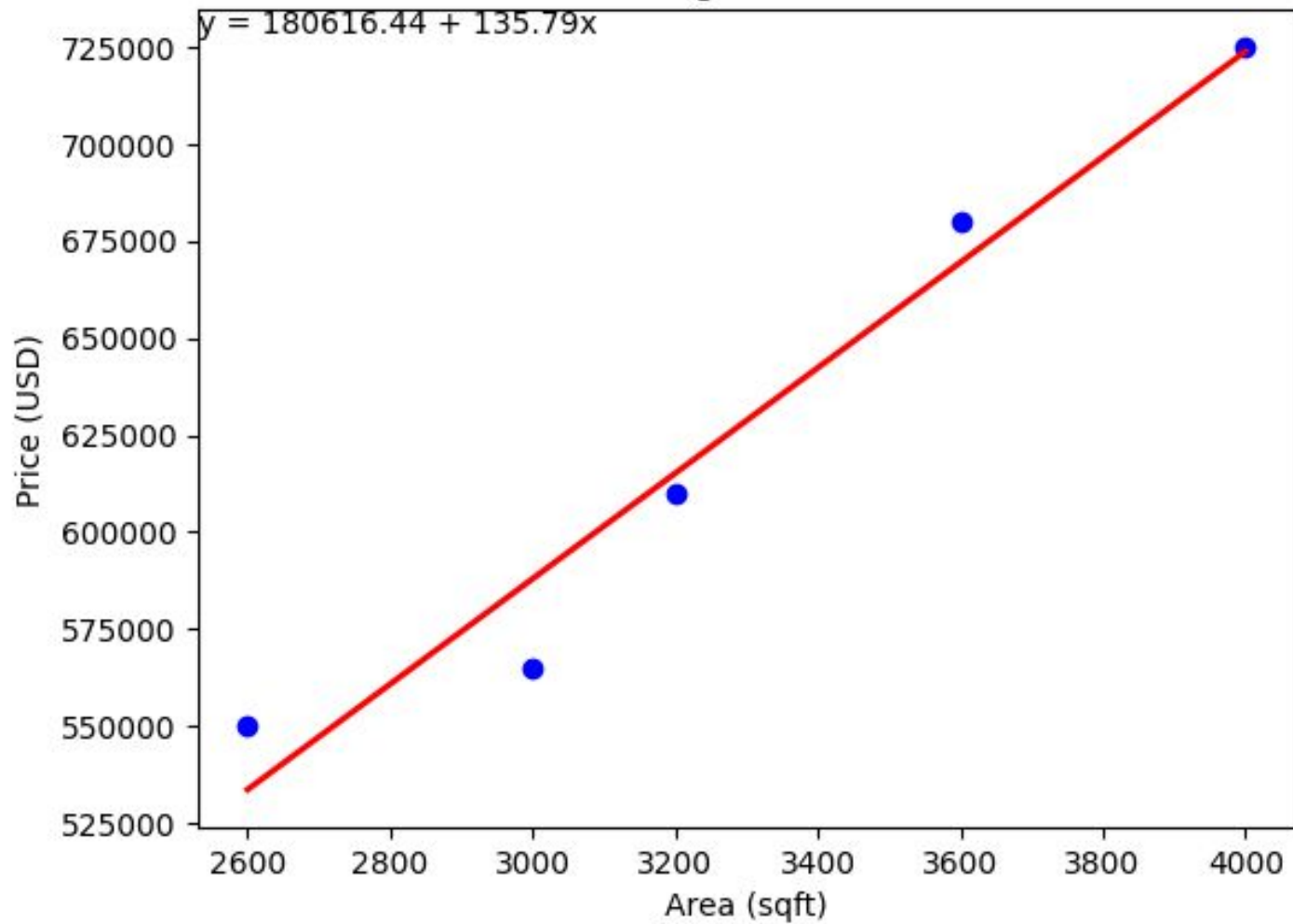


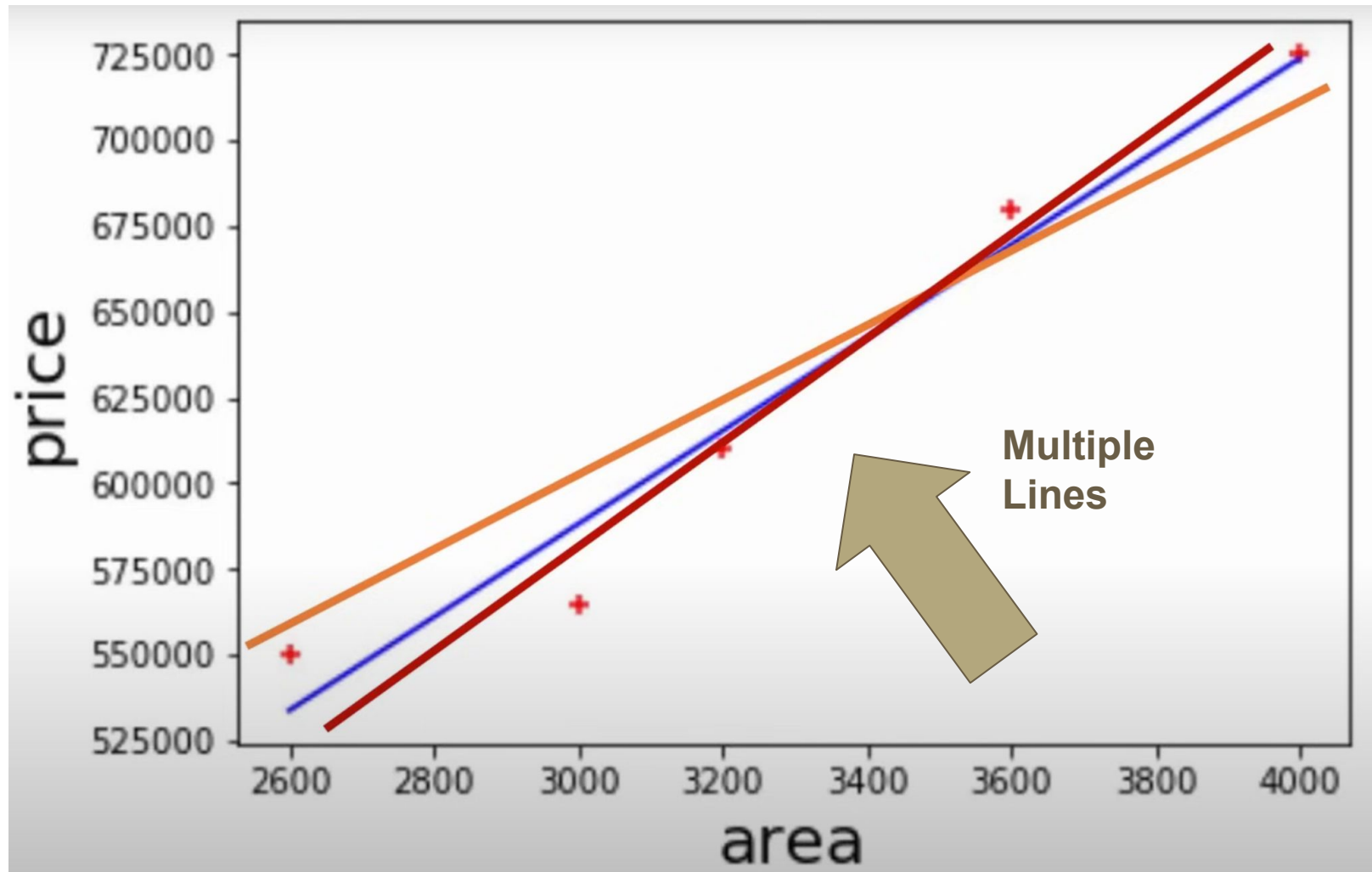
For a given Data set

area	price
2600	550000
3000	565000
3200	610000
3600	680000
4000	725000



Linear Regression Model





Mean Squared Error

$$mse = \frac{1}{n} \sum_{i=1}^n (y_i - y_{predicted})^2$$

$$mse = \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + b))^2$$

Cost Function

Gradient Descent is an algorithm that finds the best fit line for the given training data set

The main objective of using a gradient descent algorithm is to minimize the cost function using iteration. To achieve this goal, it performs two steps iteratively:

- Calculates the first-order derivative of the function to compute the gradient or slope of that function.
- Move away from the direction of the gradient, which means slope increased from the current point by α times, where α is defined as Learning Rate. It is a tuning parameter in the optimization process which helps to decide the length of the steps.

Cost Function

The cost function is defined as the measurement of difference or error between actual values and expected values at the current position and present in the form of a single real number.

It helps to increase and improve machine learning efficiency by providing feedback to this model so that it can minimize error and find the local or global minimum.

Further, it continuously iterates along the direction of the negative gradient until the cost function approaches zero. At this steepest descent point, the model will stop learning further.

Cost Function

At this steepest descent point, the model will stop learning further.

Although cost function and loss function are considered synonymous, also there is a minor difference between them.

The slight difference between the loss function and the cost function is about the error within the training of machine learning models, as loss function refers to the error of one training example, while a cost function calculates the average error across an entire training set.

The cost function is calculated after making a hypothesis with initial parameters and modifying these parameters using gradient descent algorithms over known data to reduce the cost function.

How does gradient descent works?

Before starting the working principle of gradient descent, we should know some basic concepts to find out the slope of a line from linear regression.

The equation for simple linear regression is given as:

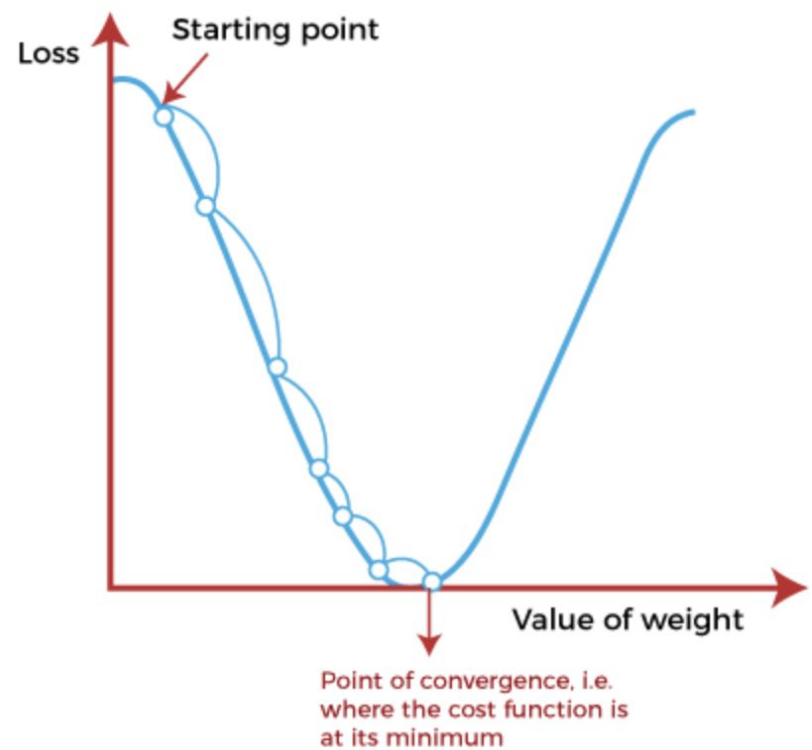
$$Y=mX+c$$

Where 'm' represents the slope of the line, and 'c' represents the intercepts on the y-axis.

The starting point (shown in fig.) is used to evaluate the performance as it is considered just as an arbitrary point. At this starting point, we will derive the first derivative or slope and then use a tangent line to calculate the steepness of this slope. Further, this slope will inform the updates to the parameters (weights and bias).

The slope becomes steeper at the starting point or arbitrary point, but whenever new parameters are generated, then steepness gradually reduces, and at the lowest point, it approaches the lowest point, which is called **a point of convergence**.

The main objective of gradient descent is to minimize the cost function or the error between expected and actual. To minimize the cost function, two data points are required:



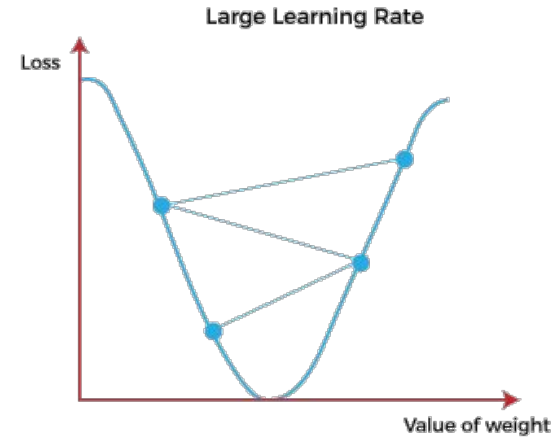
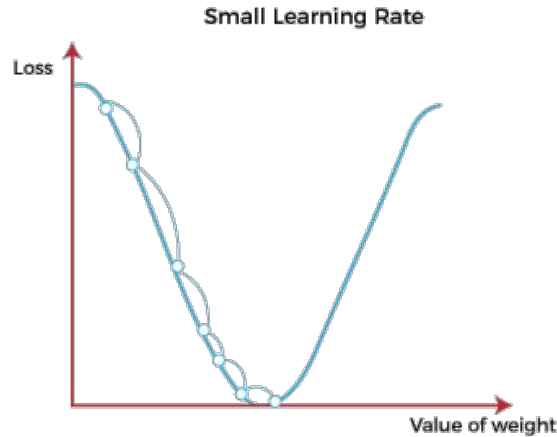
Learning Rate

It is defined as the step size taken to reach the minimum or lowest point.

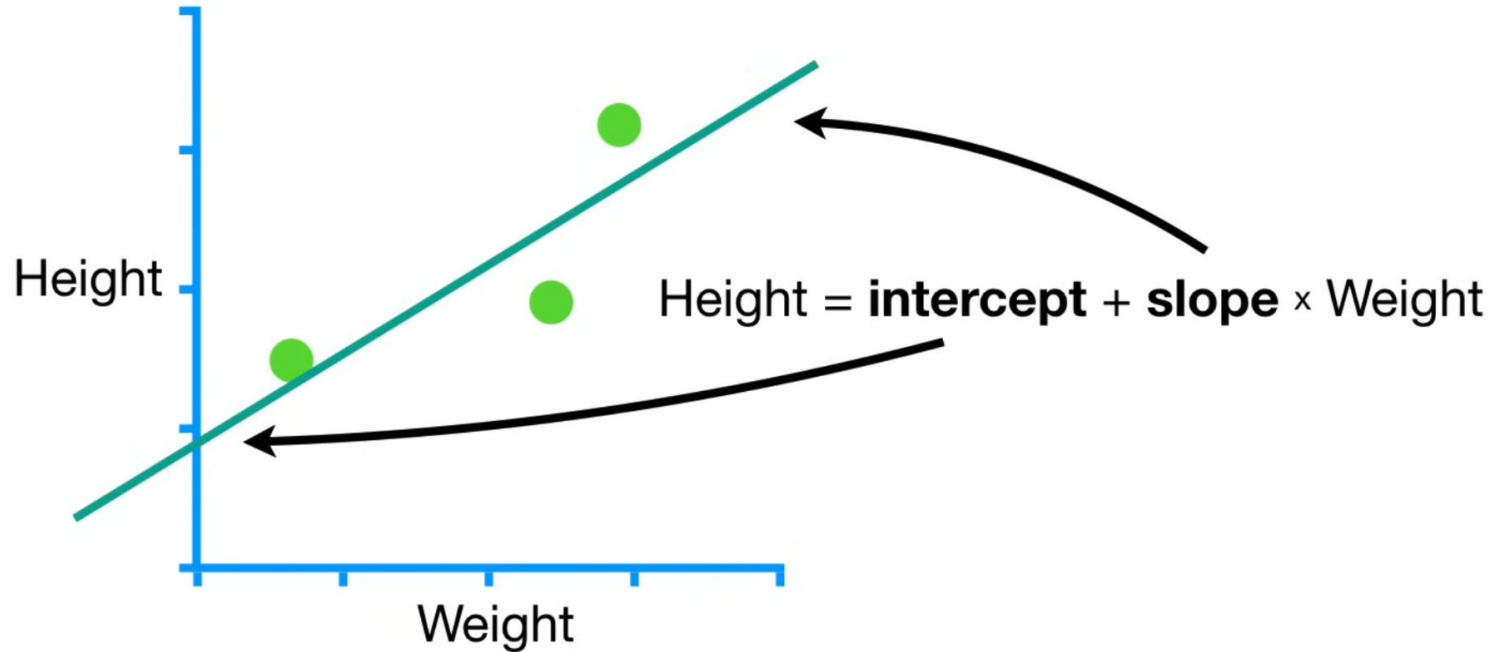
This is typically a small value that is evaluated and updated based on the behavior of the cost function.

If the learning rate is high, it results in larger steps but also leads to risks of overshooting the minimum.

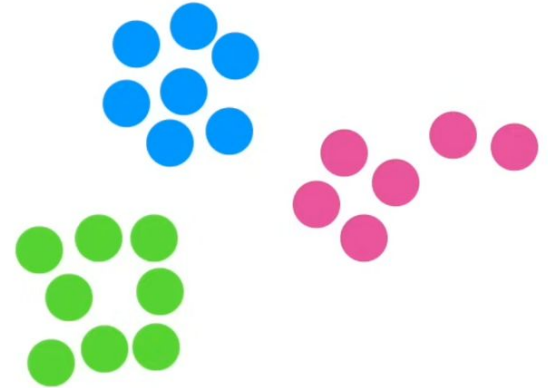
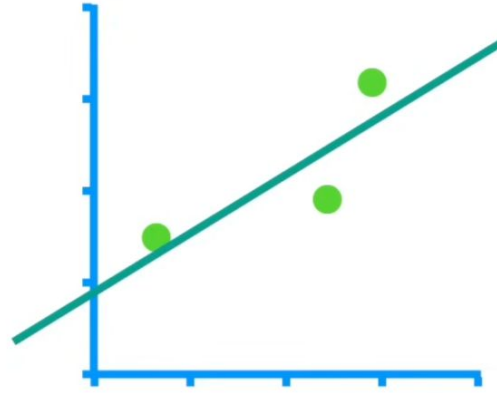
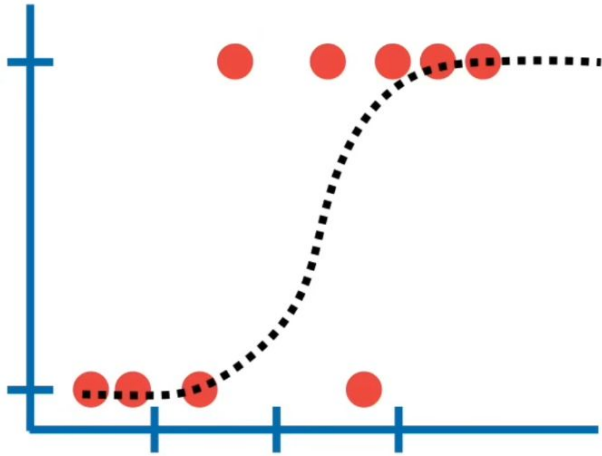
At the same time, a low learning rate shows the small step sizes, which compromises overall efficiency but gives the advantage of more precision.



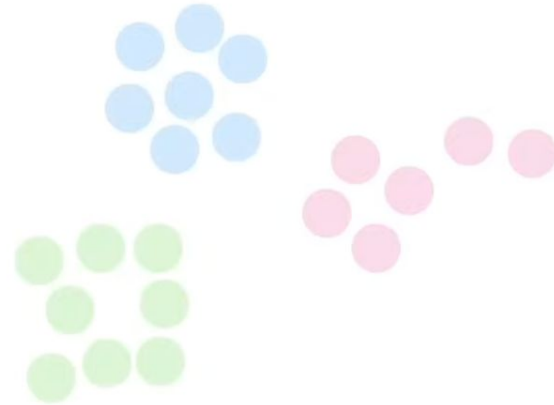
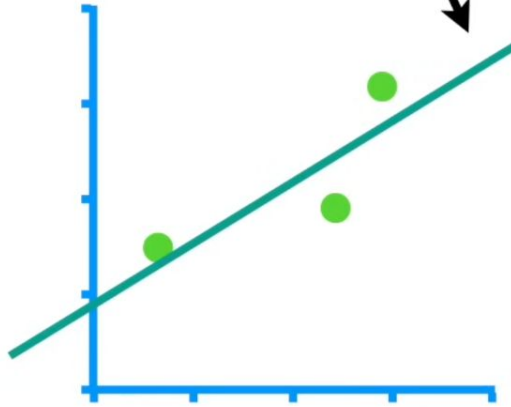
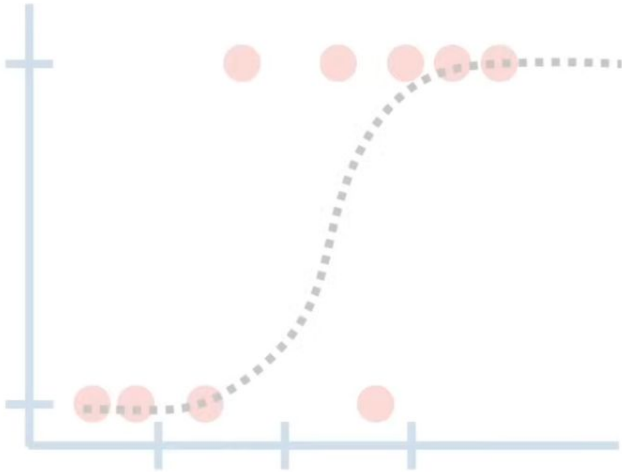
When we fit a line with **Linear Regression**, we optimize the **Intercept** and **Slope**.

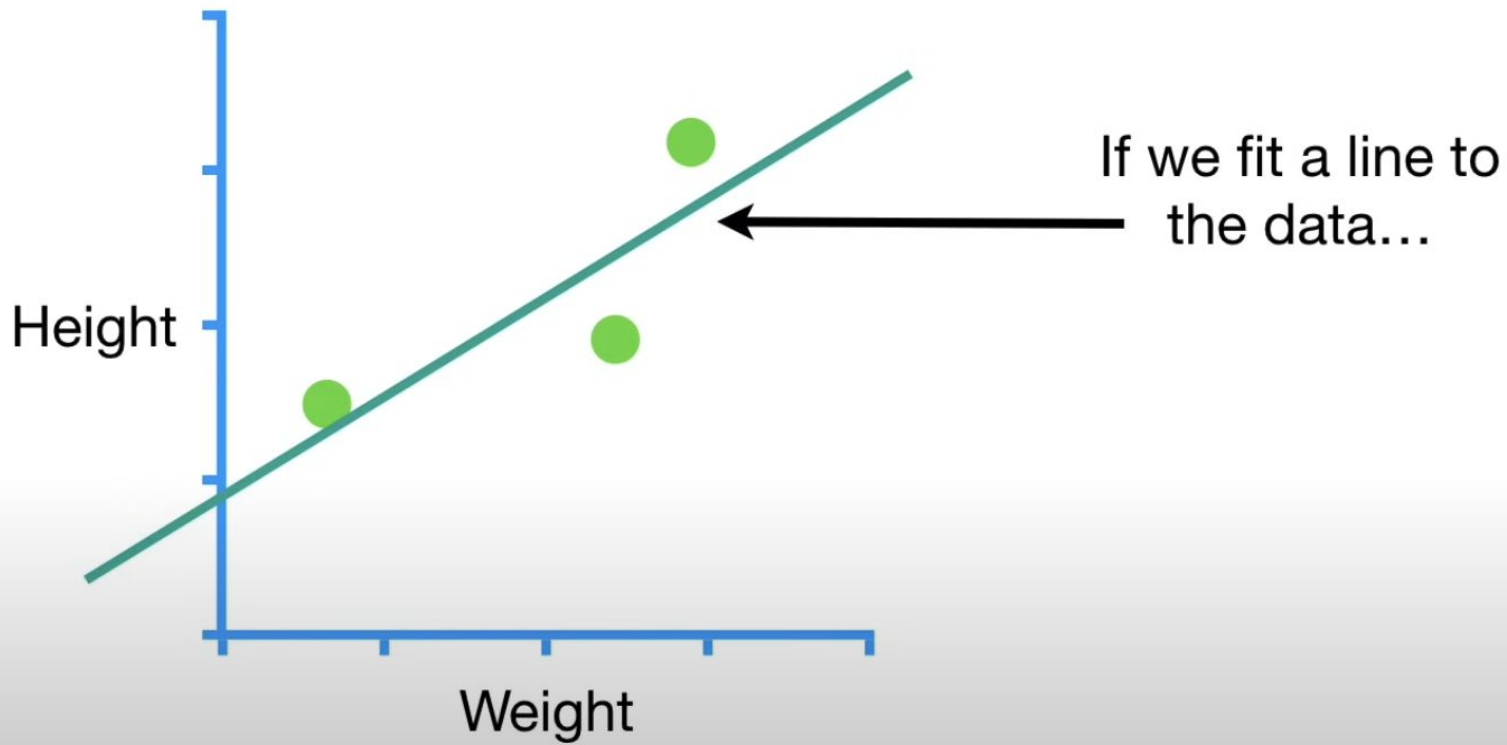


These are just a few examples of the stuff we optimize, there are tons more.

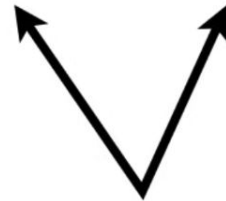


So if we learn how to optimize this line using
Gradient Descent...

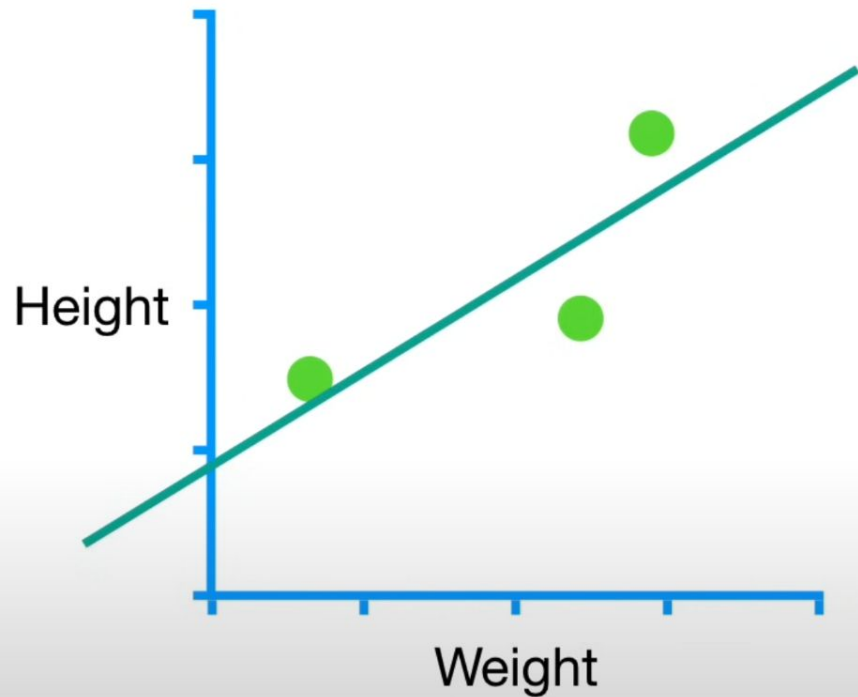


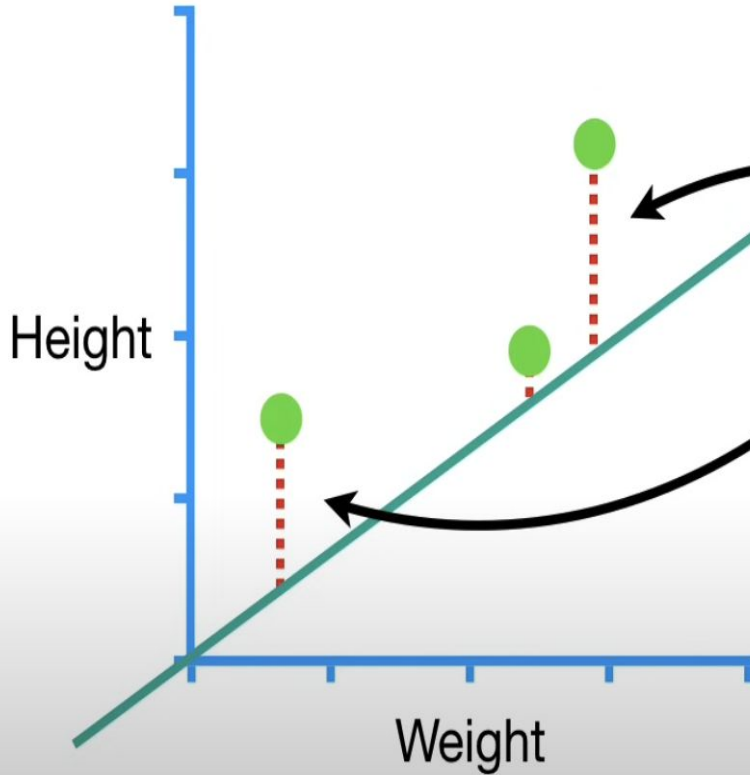


$$\text{Predicted Height} = \text{intercept} + \text{slope} \times \text{Weight}$$



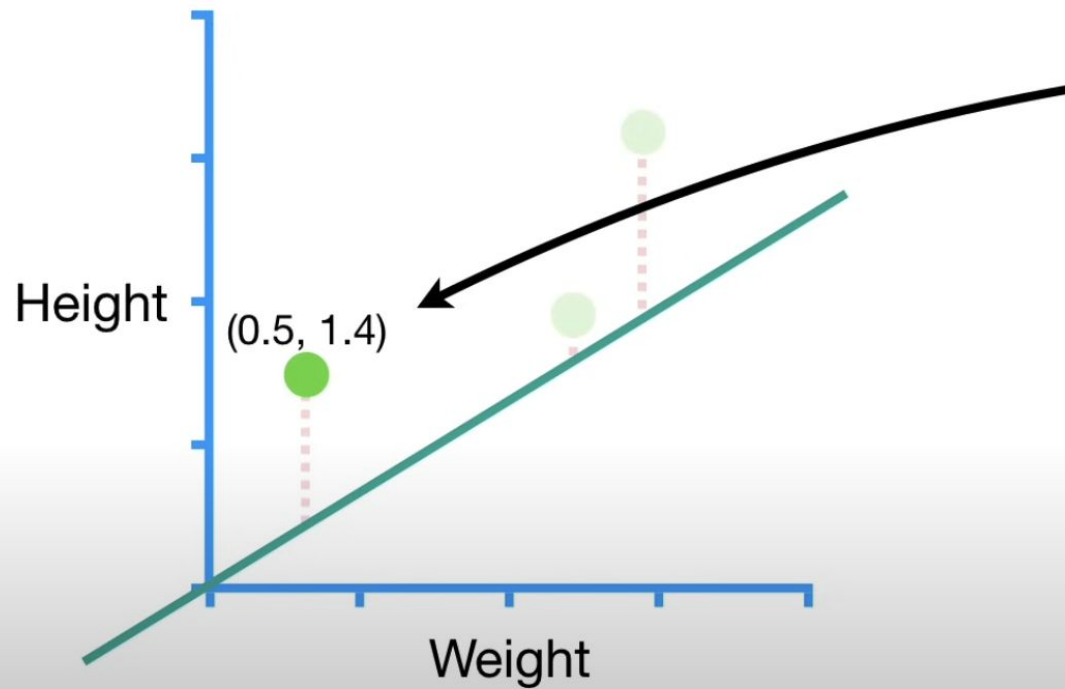
So let's learn how **Gradient Descent** can fit a line to data by finding the optimal values for the **Intercept** and the **Slope**.



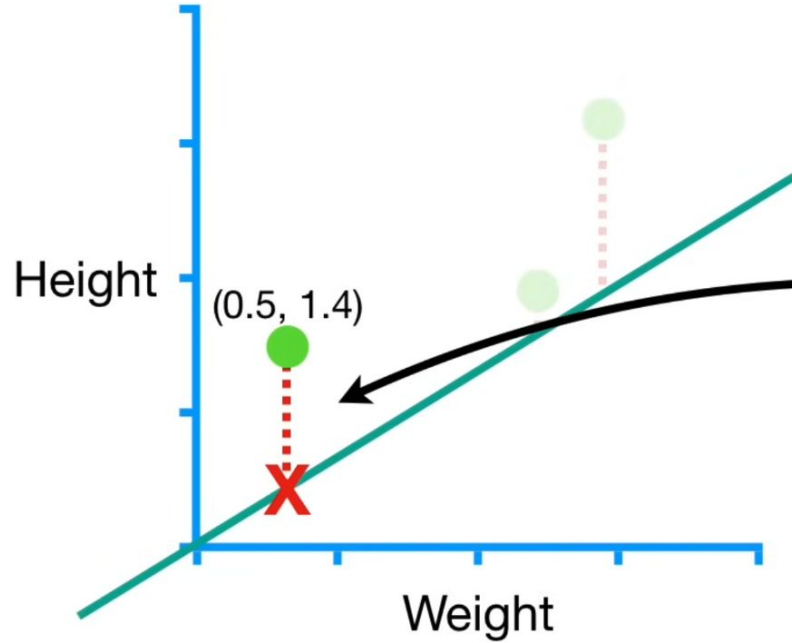


In this example, we will evaluate
how well this line fits the data
with the
Sum of the Squared Residuals.

NOTE: In Machine Learning lingo, The
Sum of the Squared Residuals is a type of
Loss Function.



This datapoint
represents a
person with
Weight 0.5 and
Height 1.4.

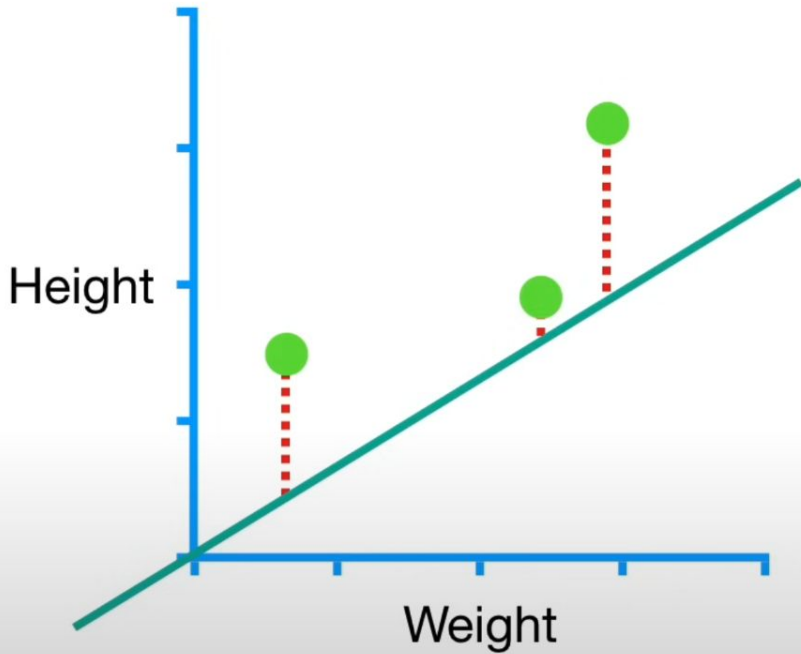


...and that gives us **1.1**
for the residual.

$$\text{Residual} = 1.4 - 0.32 = 1.1$$

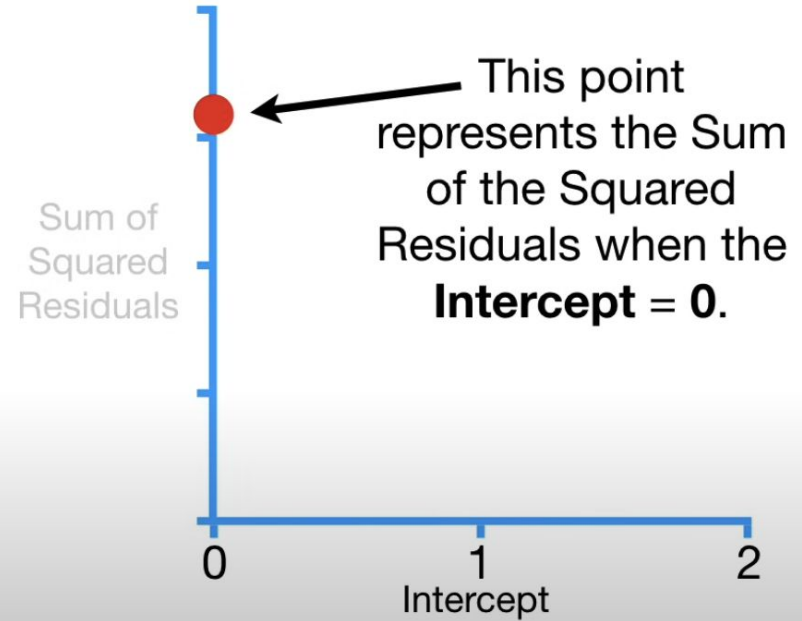
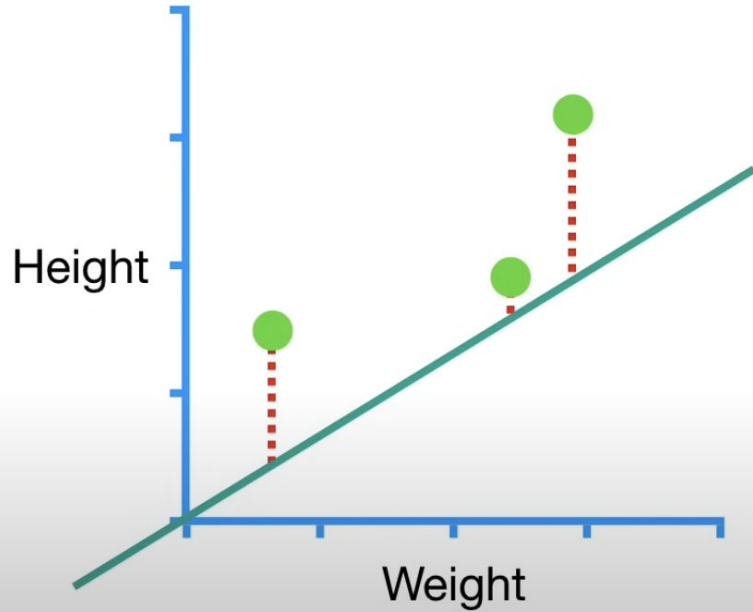
$$\text{Predicted Height} = 0 + 0.64 \times 0.5 = 0.32$$

$$\text{Sum of squared residuals} = 1.1^2 + 0.4^2 + 1.3^2 = \boxed{3.1}$$

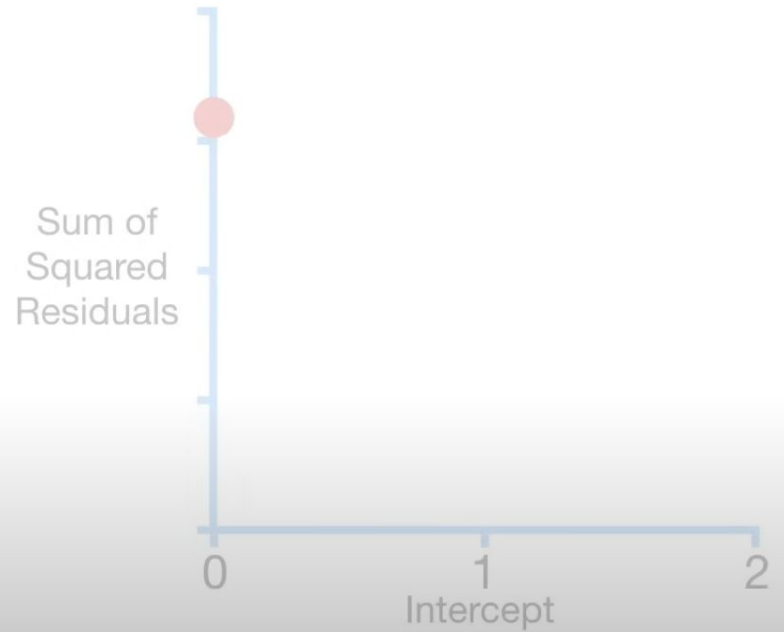
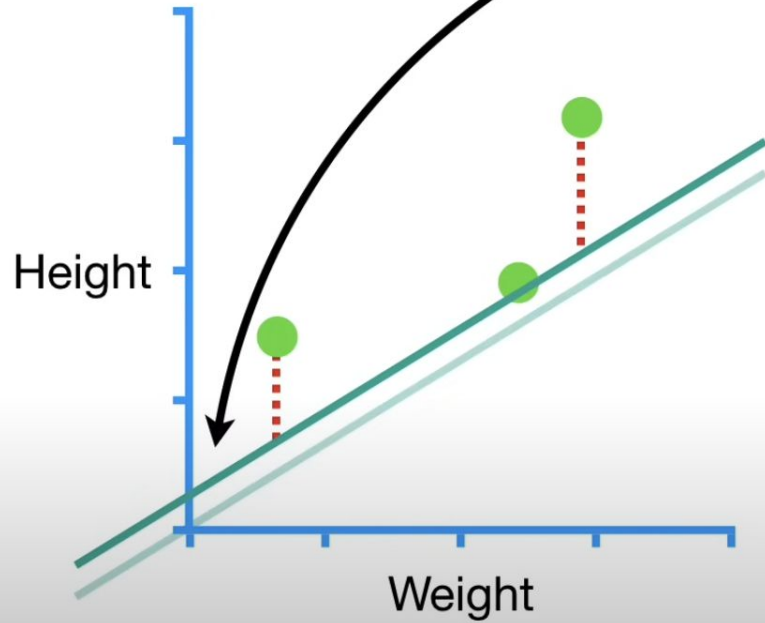


In the end, **3.1** is the Sum of the Squared Residuals.

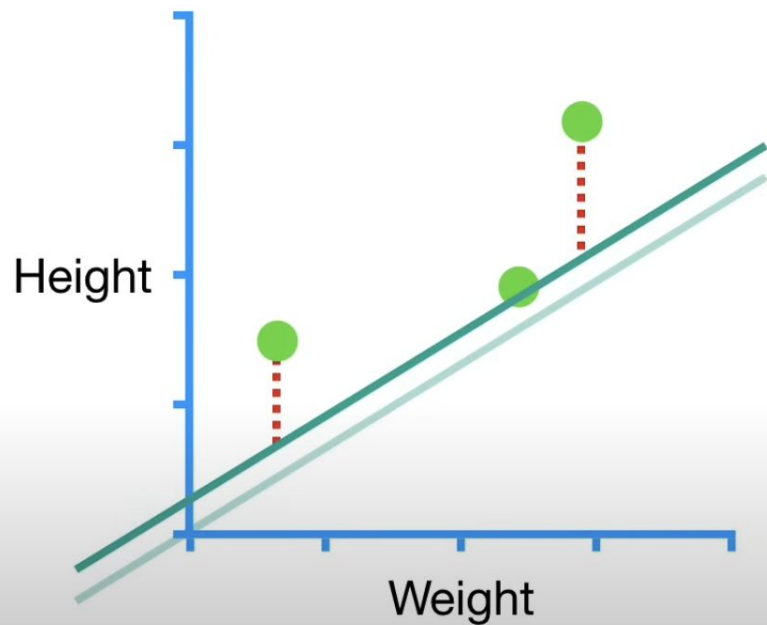
$$\text{Sum of squared residuals} = 1.1^2 + 0.4^2 + 1.3^2 = 3.1$$



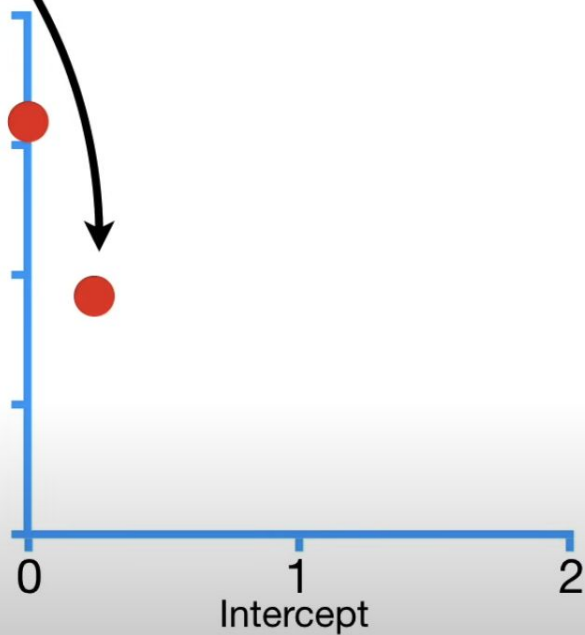
However, if the
Intercept = 0.25...



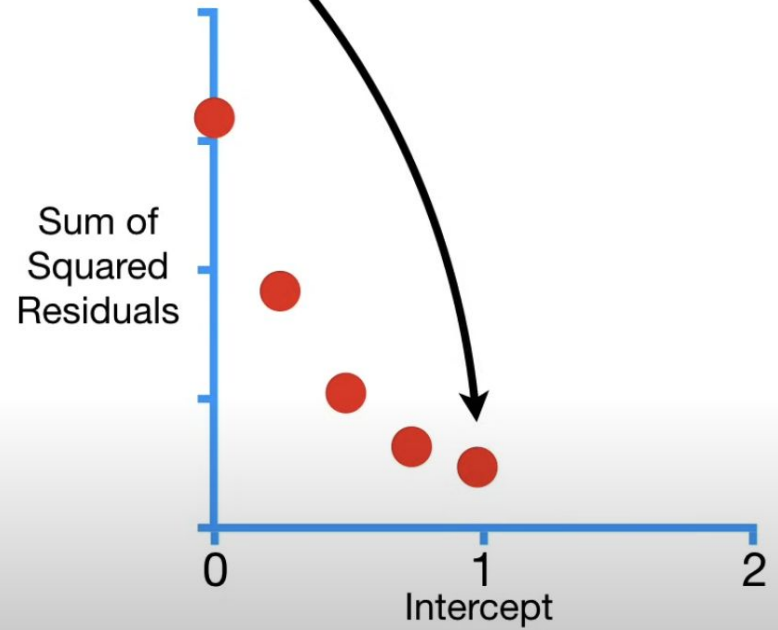
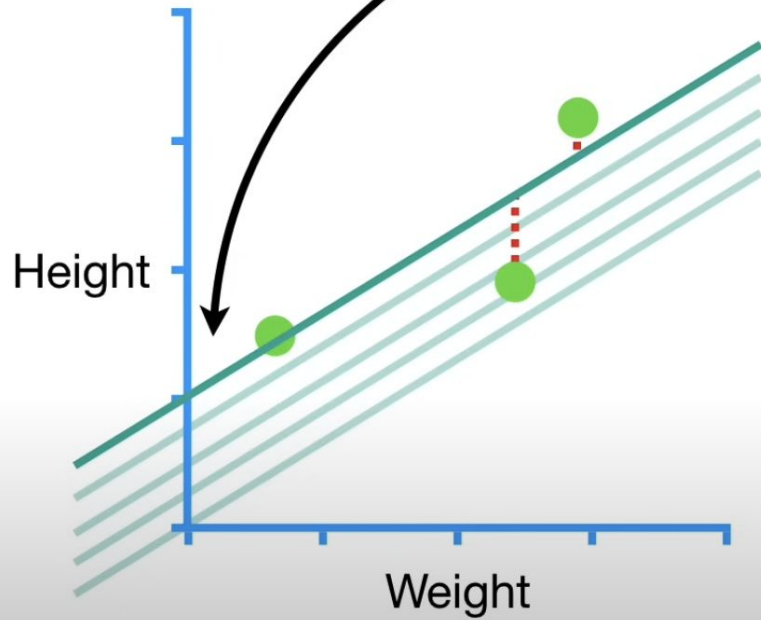
...then we would get
this point on the
graph.



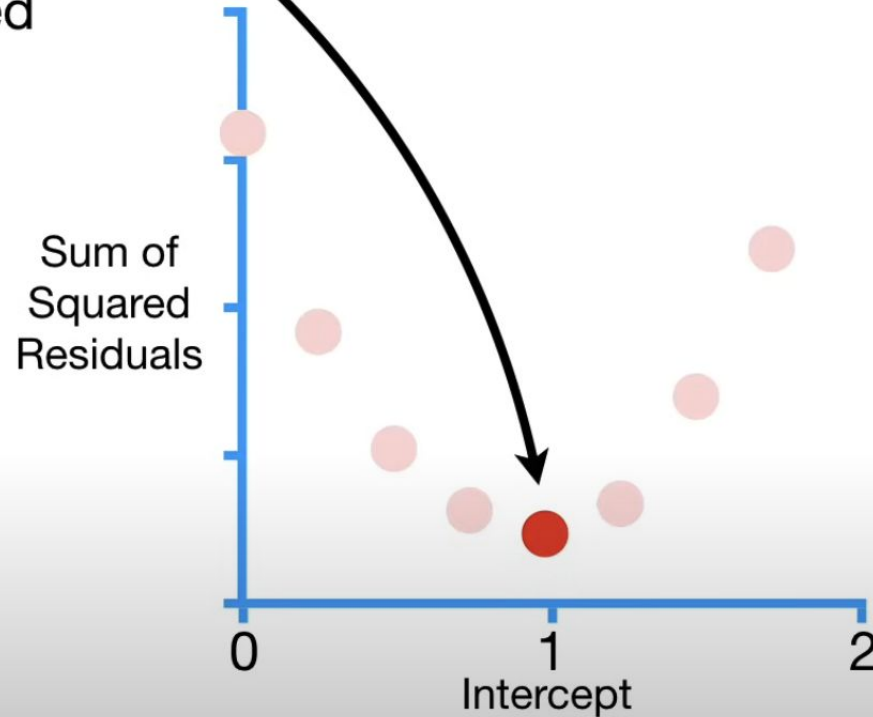
Sum of
Squared
Residuals



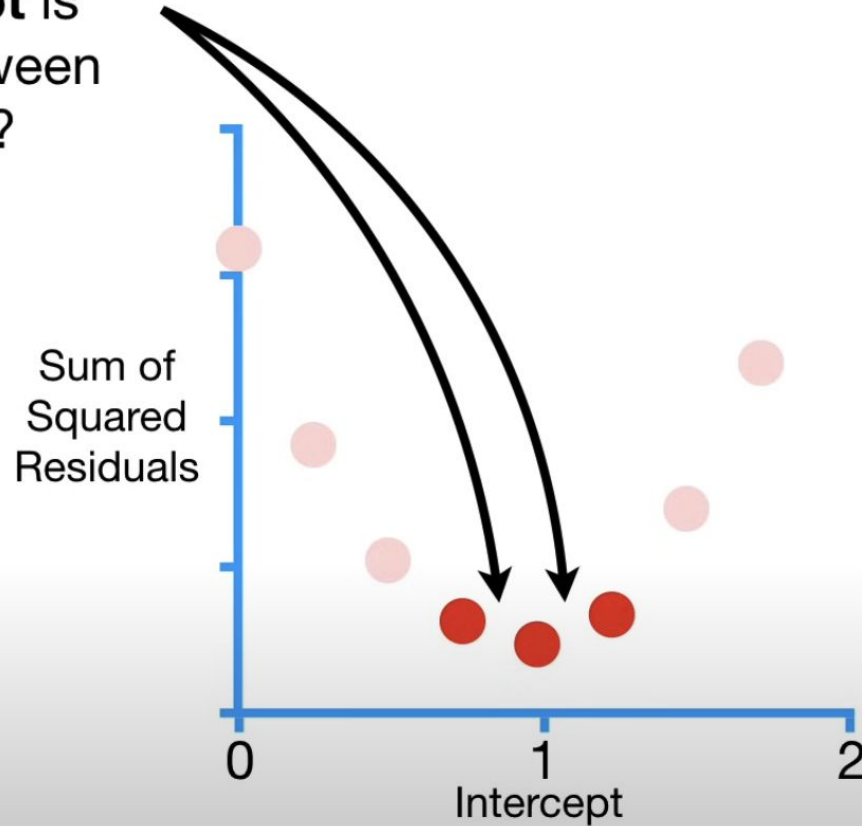
And for increasing values for the **Intercept**, we get these points.



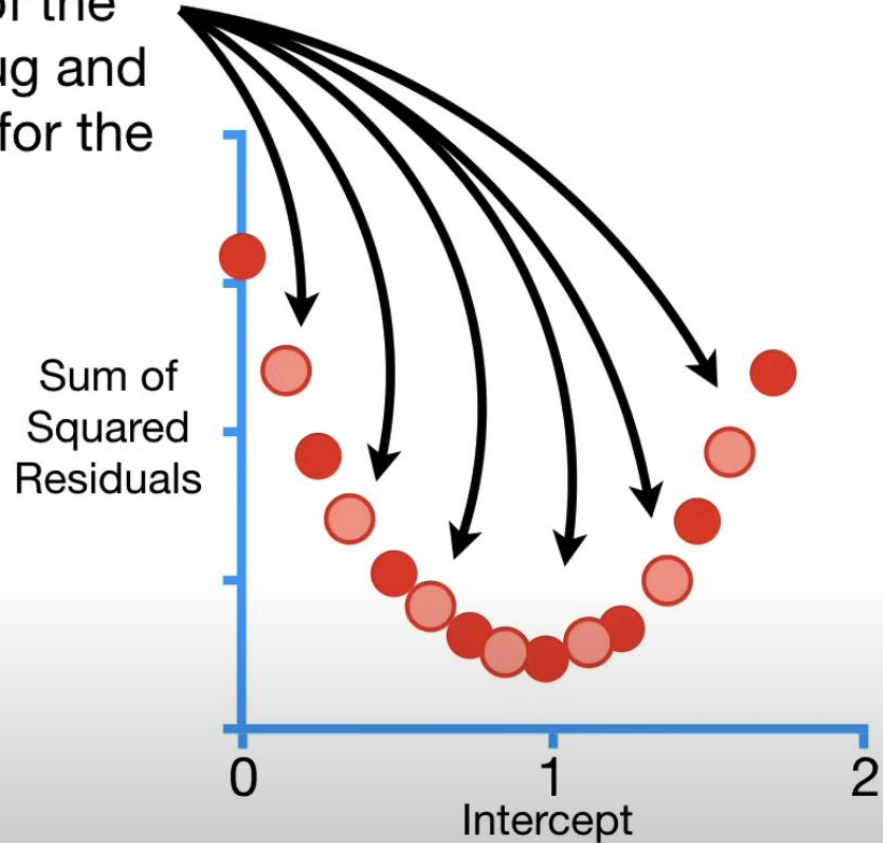
Of the points that we
calculated for the graph,
this one has the lowest
Sum of Squared
Residuals...



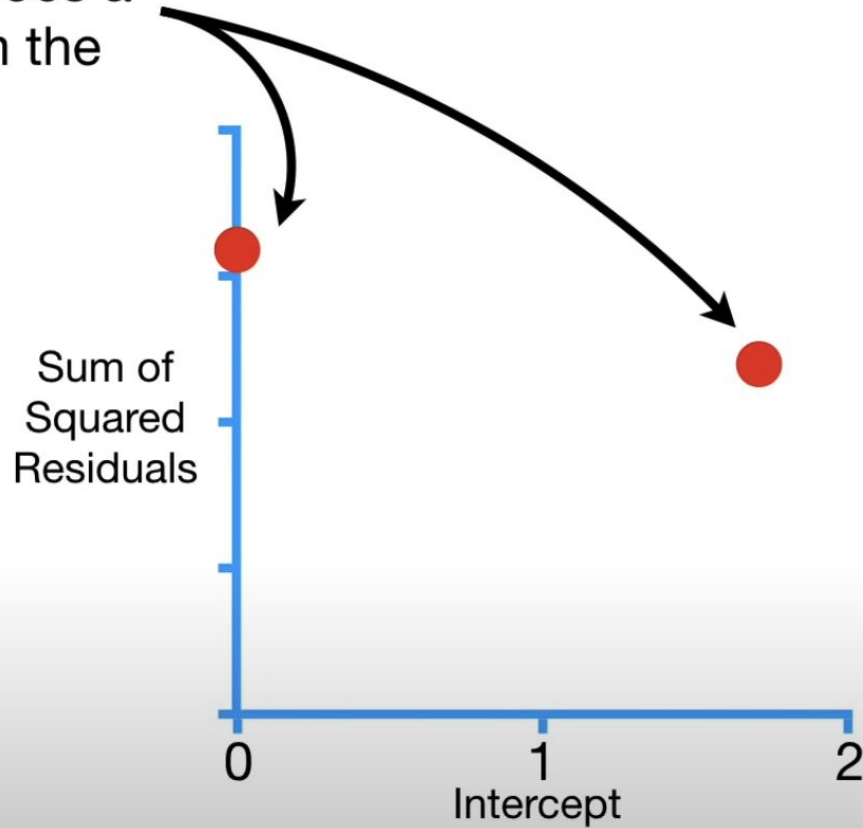
What if the best value
for the **Intercept** is
somewhere between
these values?



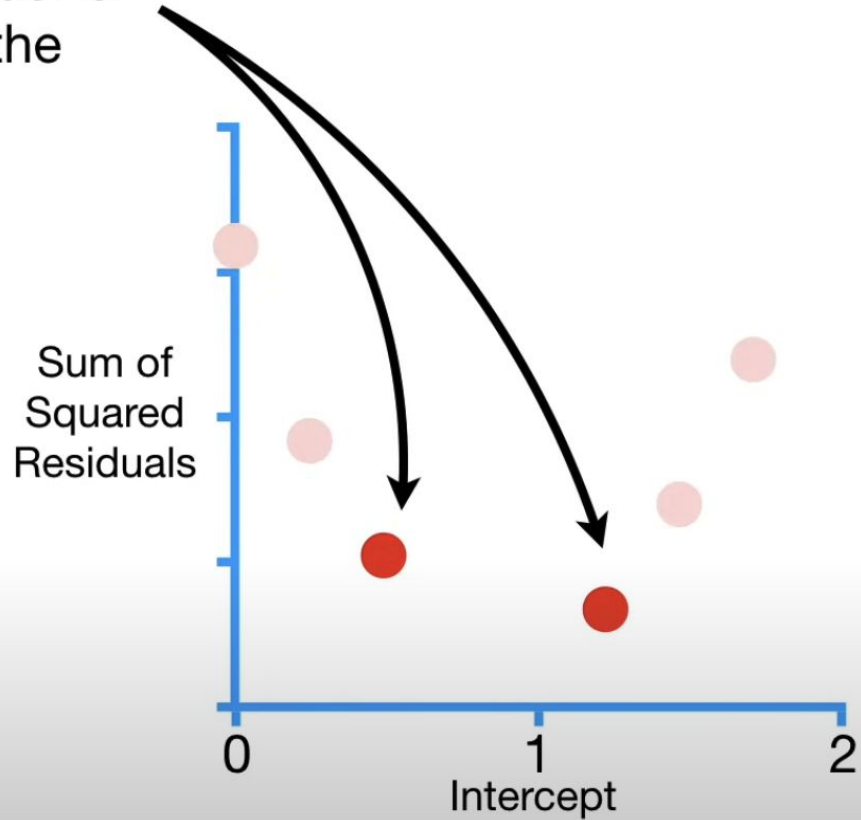
A slow and painful method for finding the minimal Sum of the Squared Residuals is to plug and chug a bunch more values for the **Intercept**.



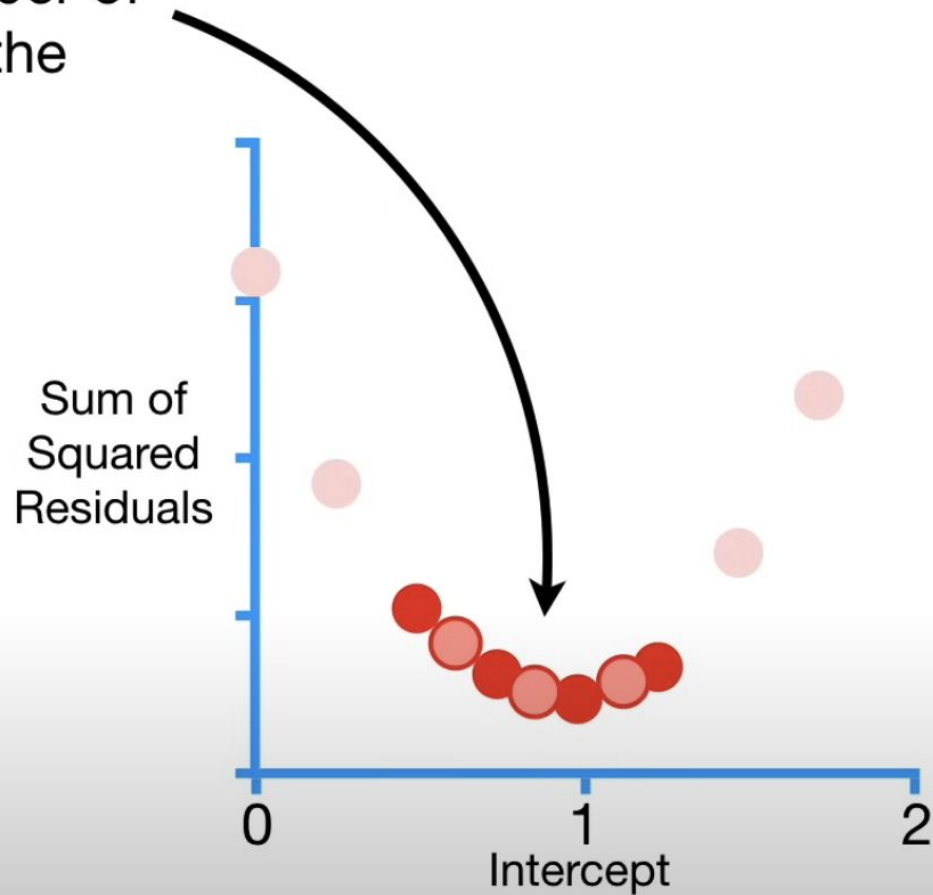
Gradient Descent only does a few calculations far from the optimal solution...



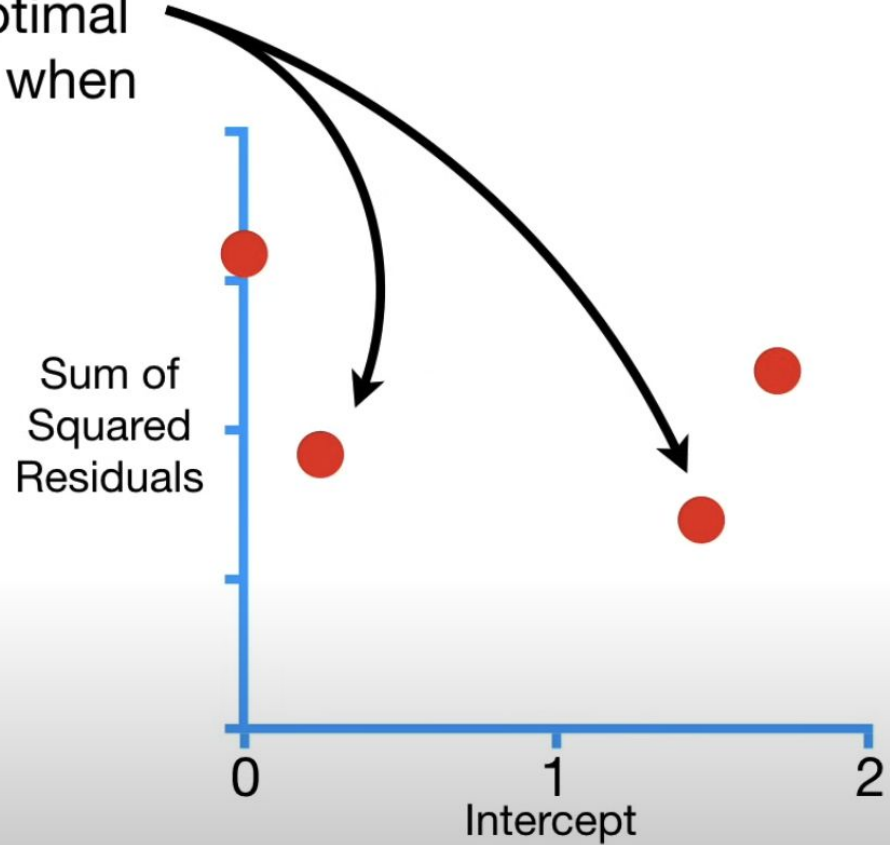
...and increases the number of calculations closer to the optimal value.



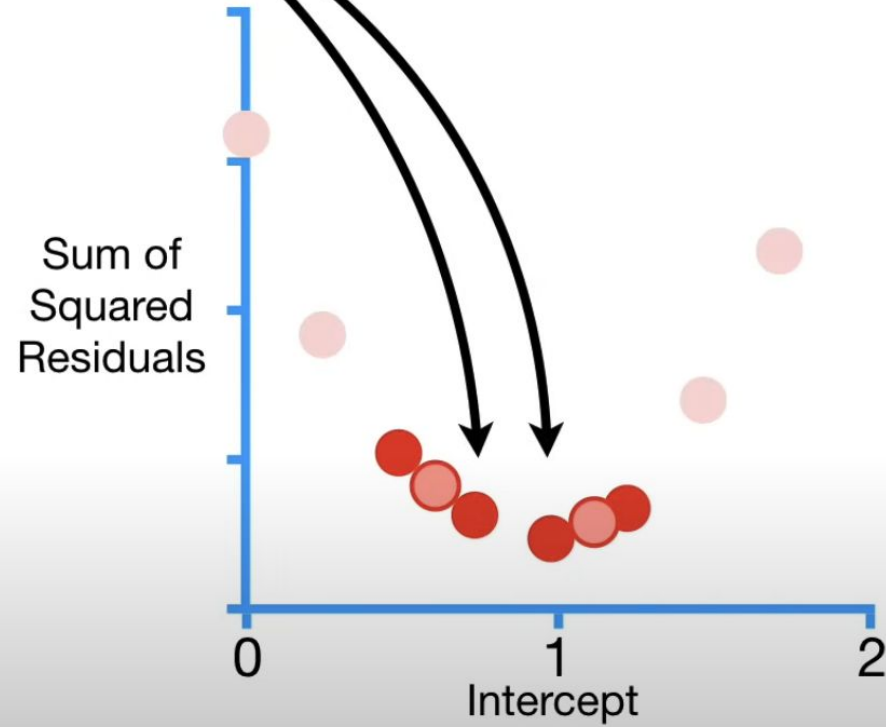
...and increases the number of calculations closer to the optimal value.



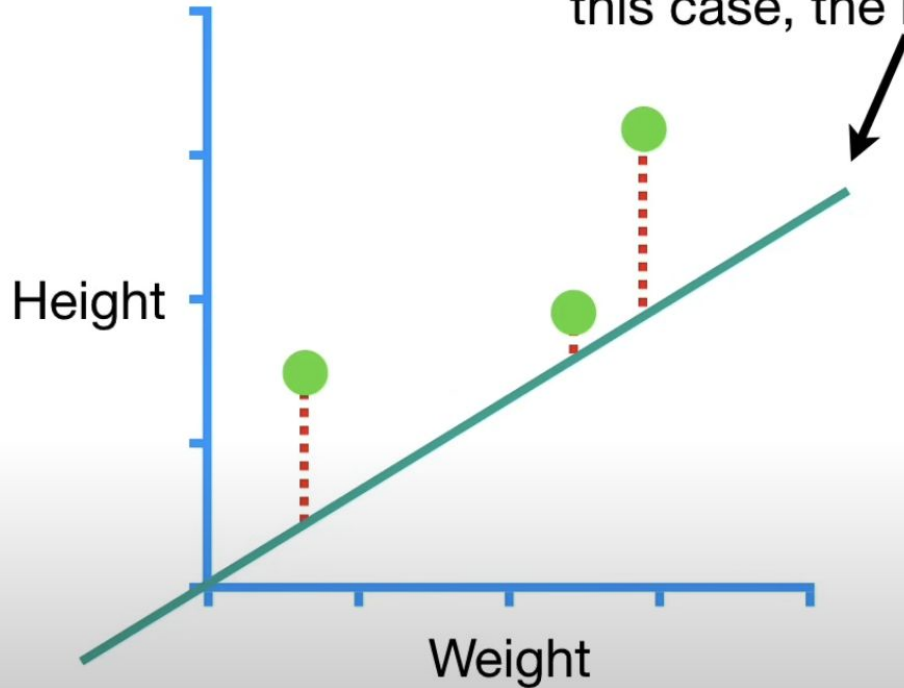
In other words, **Gradient Descent** identifies the optimal value by taking big steps when it is far away...



...and baby steps
when it is close.

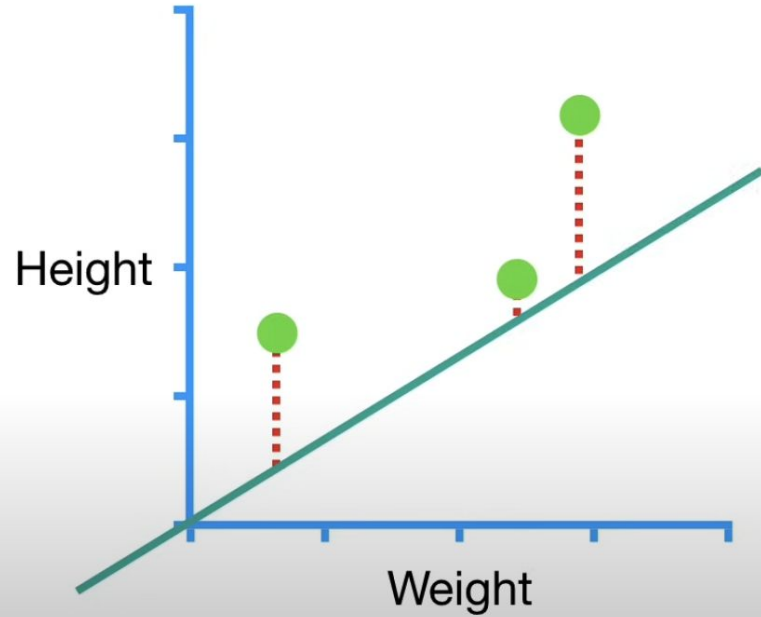


So let's get back to using **Gradient Descent** to find the optimal value for the **Intercept**, starting from a random value. In this case, the random value was **0**.



Sum of squared residuals

When we calculated the
Sum of the Squared
Residuals...



Sum of
Squared
Residuals

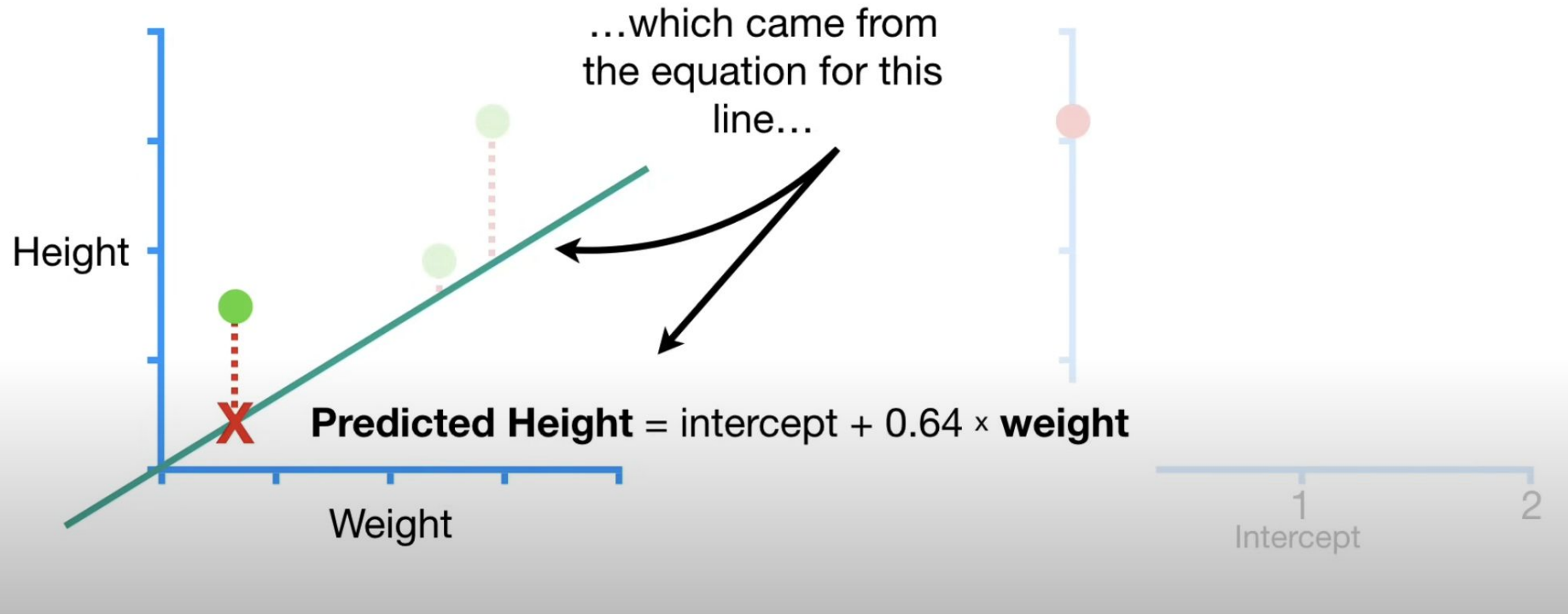
0

1

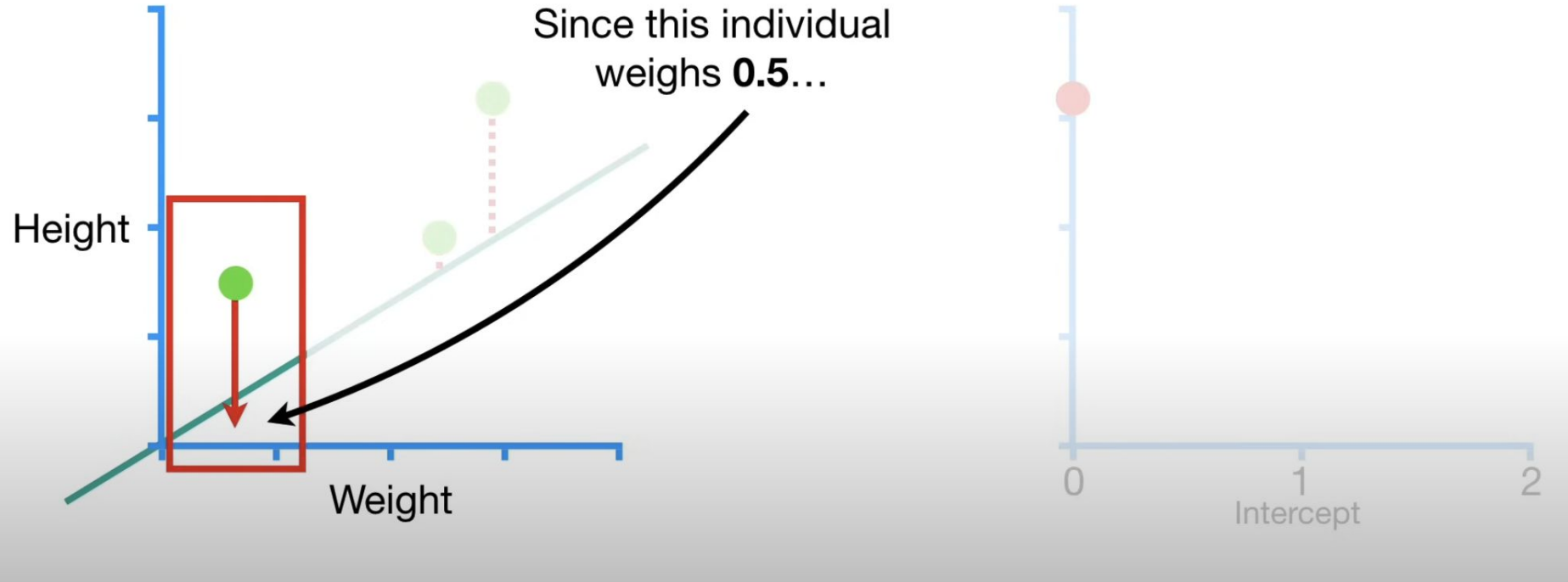
2

Intercept

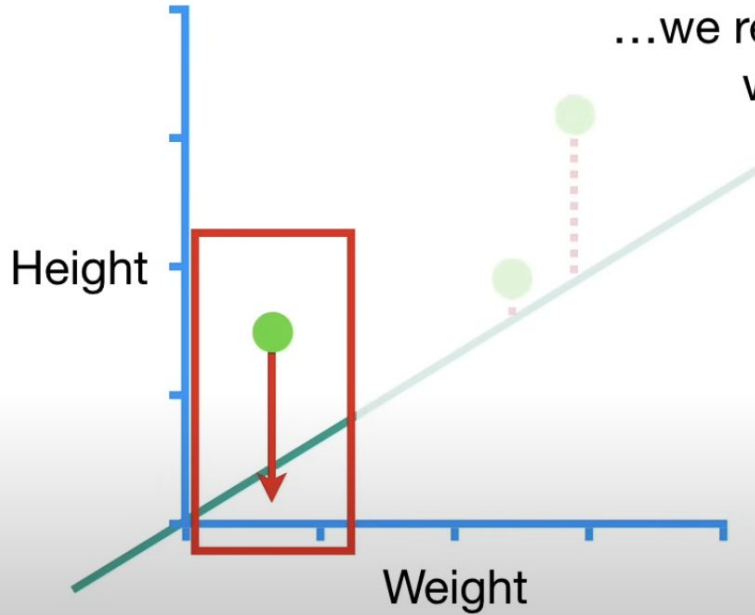
Sum of squared residuals = $(1.4 - \text{predicted})^2$



Sum of squared residuals = $(1.4 - (\text{intercept} + 0.64 \times \text{weight}))^2$



Sum of squared residuals = $(1.4 - (\text{intercept} + 0.64 \times 0.5))^2$



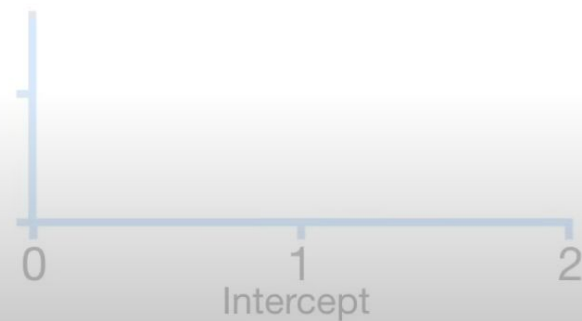
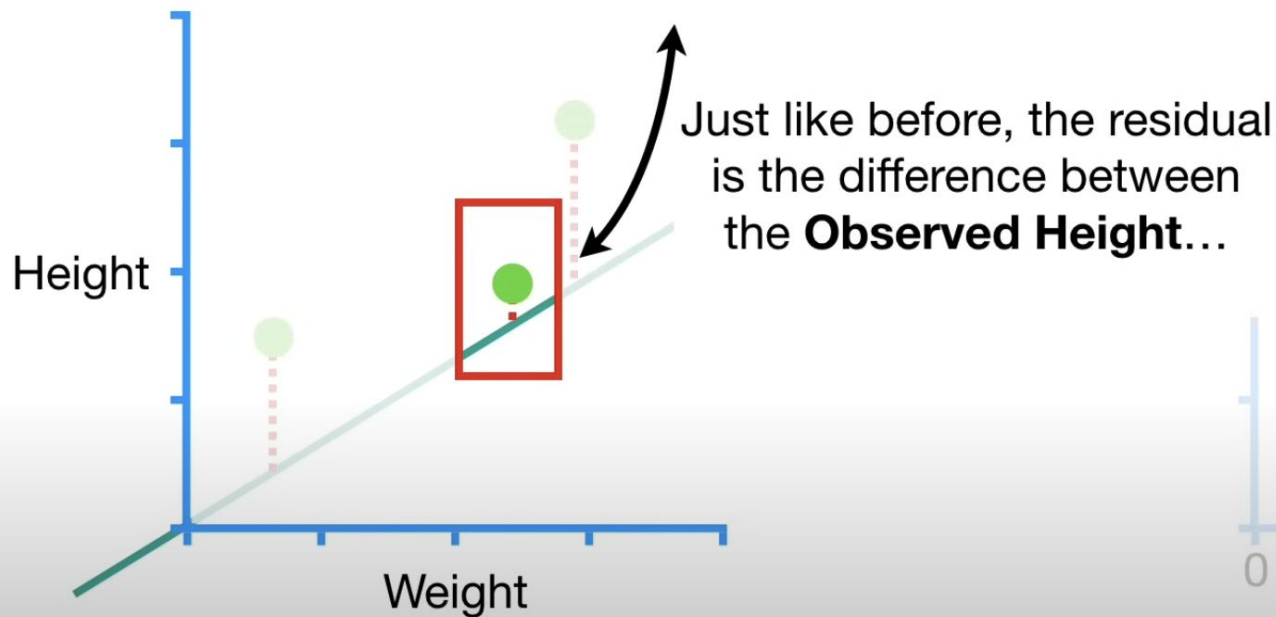
...we replace **weight**
with **0.5**.



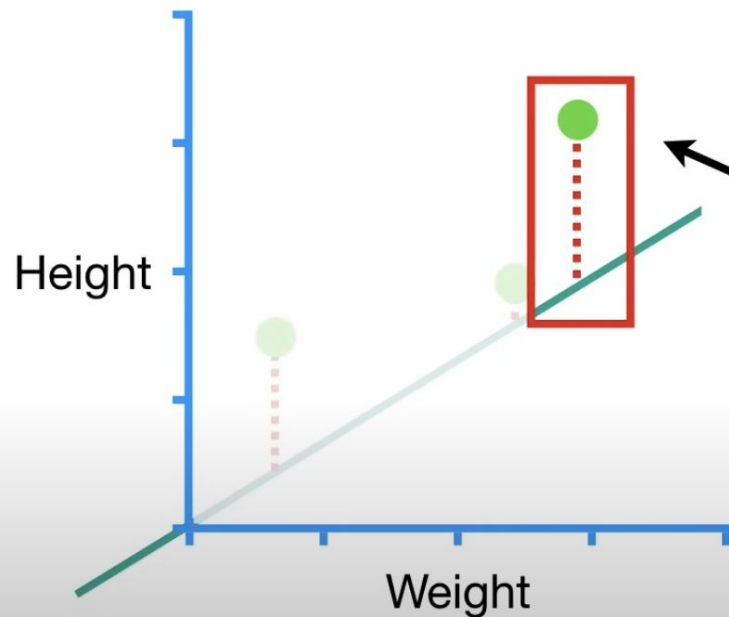
Sum of squared residuals = $(1.4 - (\text{intercept} + 0.64 \times 0.5))^2$



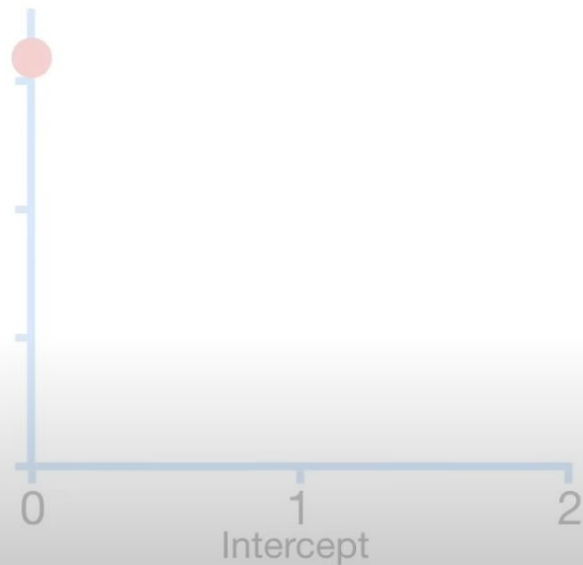
$$\text{Sum of squared residuals} = (1.4 - (\text{intercept} + 0.64 \times 0.5))^2 + (\text{observed} - \text{predicted})^2$$



$$\begin{aligned}\text{Sum of squared residuals} &= (1.4 - (\text{intercept} + 0.64 \times \mathbf{0.5}))^2 \\ &\quad + (1.9 - (\text{intercept} + 0.64 \times \mathbf{2.3}))^2\end{aligned}$$



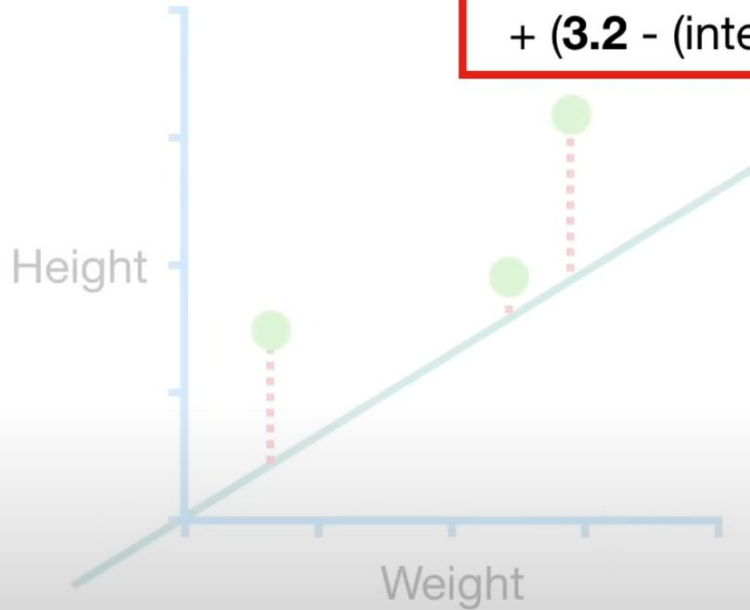
Now let's focus on
the last person.



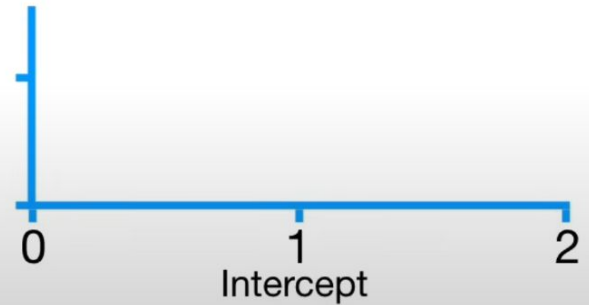
Sum of squared residuals = $(1.4 - (\text{intercept} + 0.64 \times 0.5))^2$

+ $(1.9 - (\text{intercept} + 0.64 \times 2.3))^2$

+ $(3.2 - (\text{intercept} + 0.64 \times 2.9))^2$



Now we can easily
plug in any value for
the **intercept**...



$$\begin{aligned}\text{Sum of squared residuals} = & (1.4 - (\text{intercept} + 0.64 \times 0.5))^2 \\ & + (1.9 - (\text{intercept} + 0.64 \times 2.3))^2 \\ & + (3.2 - (\text{intercept} + 0.64 \times 2.9))^2\end{aligned}$$

Height

Weight

...and get the **Sum
of the Squared
Residuals.**

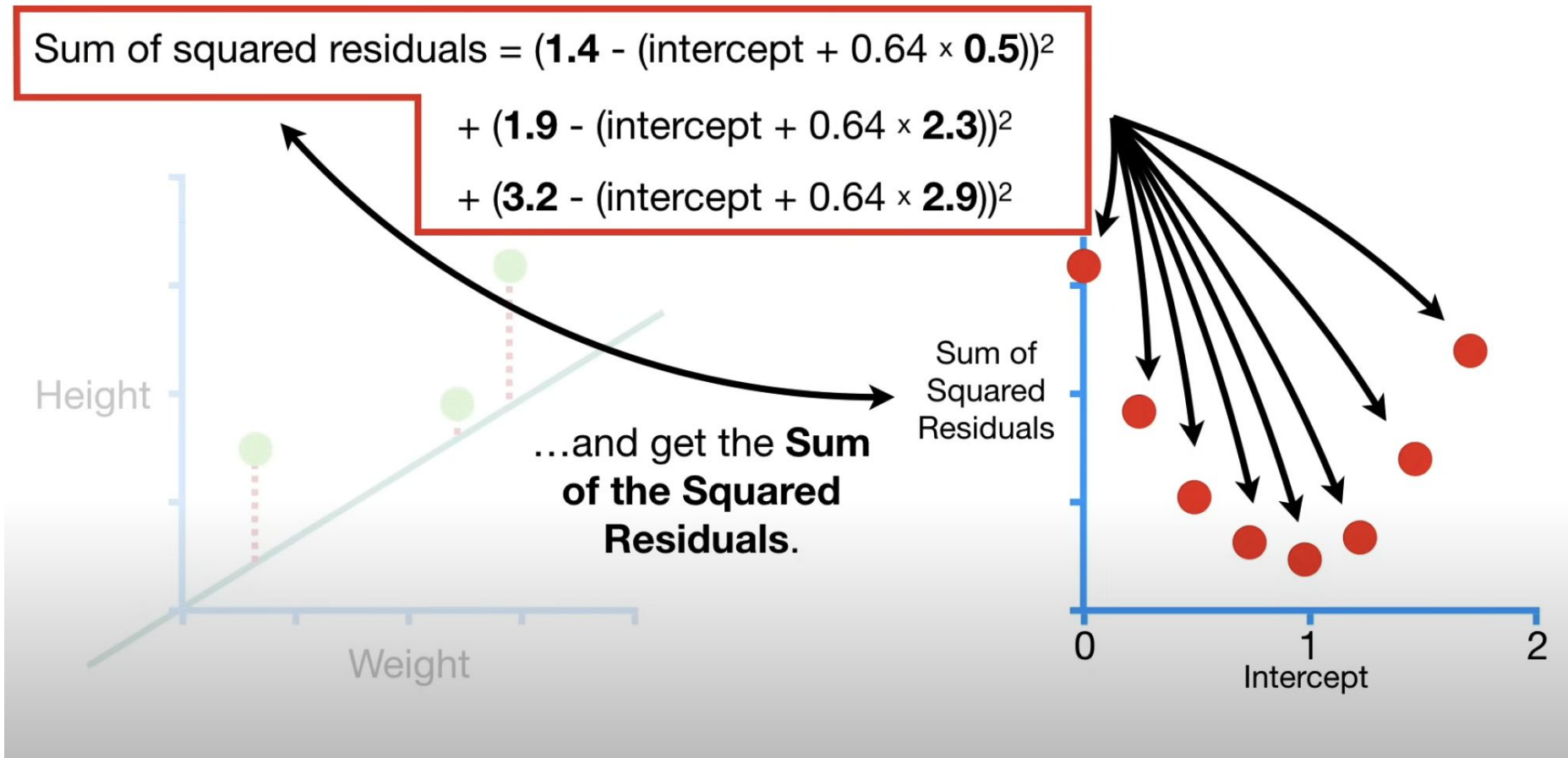
Sum of
Squared
Residuals

0

1

2

Intercept

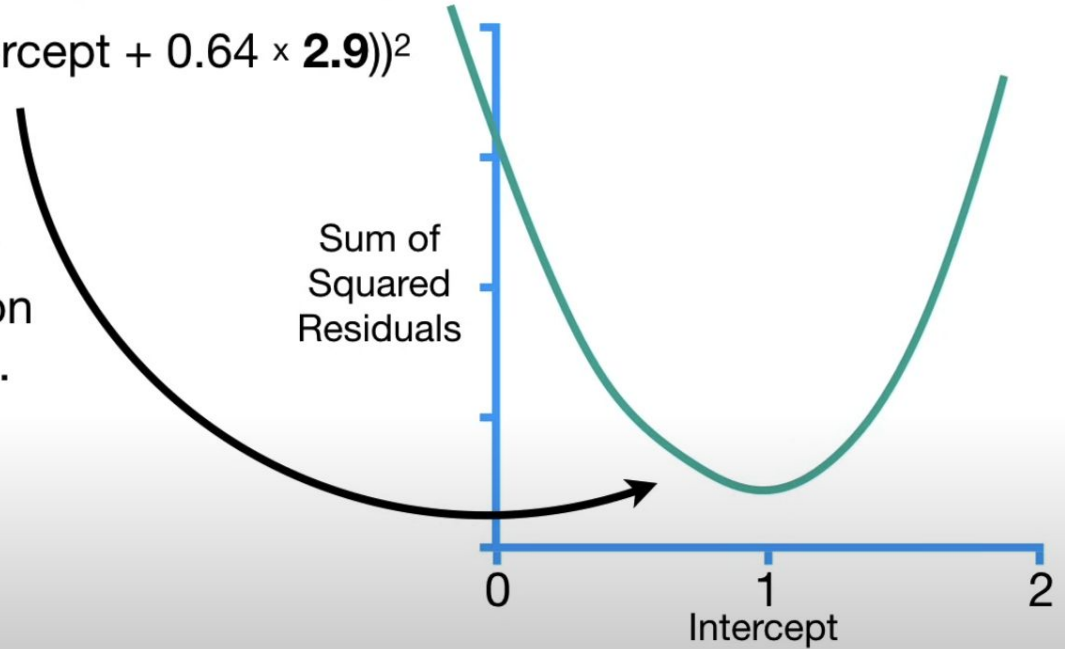


Sum of squared residuals = $(1.4 - (\text{intercept} + 0.64 \times 0.5))^2$

+ $(1.9 - (\text{intercept} + 0.64 \times 2.3))^2$

+ $(3.2 - (\text{intercept} + 0.64 \times 2.9))^2$

Thus, we now
have an equation
for this curve...

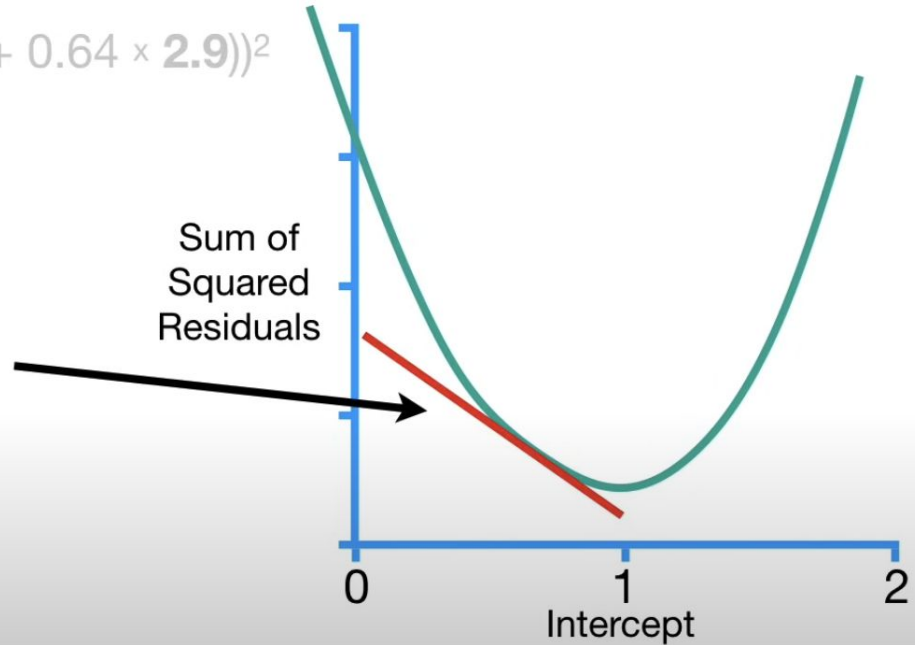


Sum of squared residuals = $(1.4 - (\text{intercept} + 0.64 \times 0.5))^2$

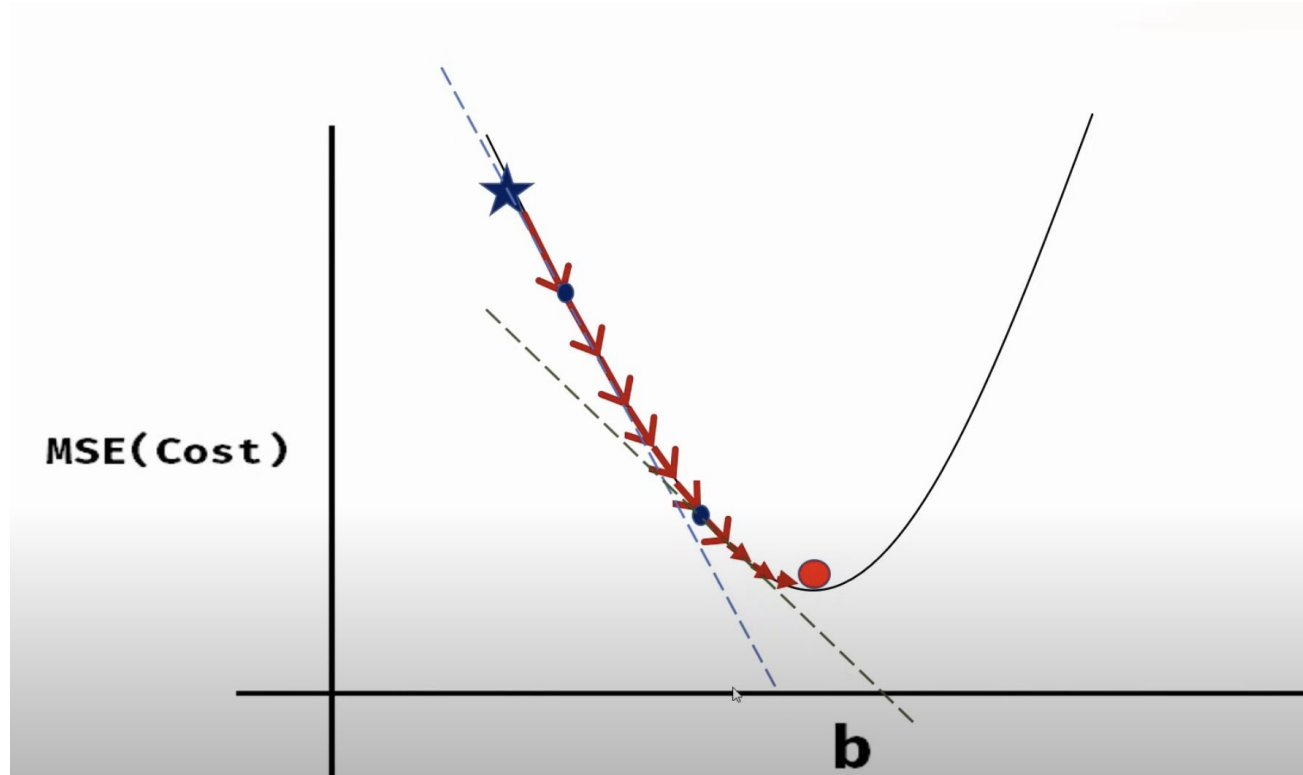
+ $(1.9 - (\text{intercept} + 0.64 \times 2.3))^2$

+ $(3.2 - (\text{intercept} + 0.64 \times 2.9))^2$

...and we can take the derivative of this function and determine the slope at any value for the **Intercept**.



Need to calculate



Finding the derivatives

$$mSE = \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + b))^2$$

$$\partial / \partial m = \frac{2}{n} \sum_{i=1}^n -x_i (y_i - (mx_i + b))$$

$$\partial / \partial b = \frac{2}{n} \sum_{i=1}^n -(y_i - (mx_i + b))$$

Calculation of slope and intercept



Using partial derivative

$$m = m - \text{learning rate} * \frac{\partial}{\partial m}$$

$$b = b - \text{learning rate} * \frac{\partial}{\partial b}$$

For the given dataset, try to fit the linear regression.

x	y
1	2
2	3
3	5
4	7

Try to fit in the linear regression:

$$y = a + bx$$

Initialize the guesses: $m=0$, $b=0$

Initialize the learning rate (α) = 0.01

Cost Function:

$$MSE = 1/n * (\sum (Actual - Predicted)^2)$$

Compute the gradients using the formula:

$$\partial m / \partial J = -2/n * \sum (x_i (y_i - (m * x_i + b)))$$

$$\partial J / \partial b = -2/n * \sum (y_i - (m * x_i + b))$$

Update the gradients:

$$m := m - \eta * \partial m / \partial J$$

$$b := b - \eta * \partial J / \partial b$$

Iteration 1:

For each data point, For each data point, compute

$$y' = mx + b = 0x + 0 = 0$$

Calculate Errors:

- For $x=1, y=2: e=2-0=2$
- For $x=2, y=3: e=3-0=3$
- For $x=3, y=5: e=5-0=5$
- For $x=4, y=7: e=7-0=7$

Compute Gradients:

Average error for m :

$$\partial J / \partial m = -2/4(1*2+2*3+3*5+4*7) = -2/4(2+6+15+28) = -2/4(51) = -25.5$$

Average error for b :

$$\partial J / \partial b = -2/4(2+3+5+7) = -2/4(17) = -8.5$$

Update Parameters

- Update m :

$$m := m - (0.01)(-25.5) = 0 + 0.255 = 0.255$$

- Update b :

$$b := b - (0.01)(-8.5) = 0 + 0.085 = 0.085$$

Repeat this iteration for about say 100 times, they will converge to a point.

Suppose,

$$m \approx 1.75$$

$$b \approx 0.25$$

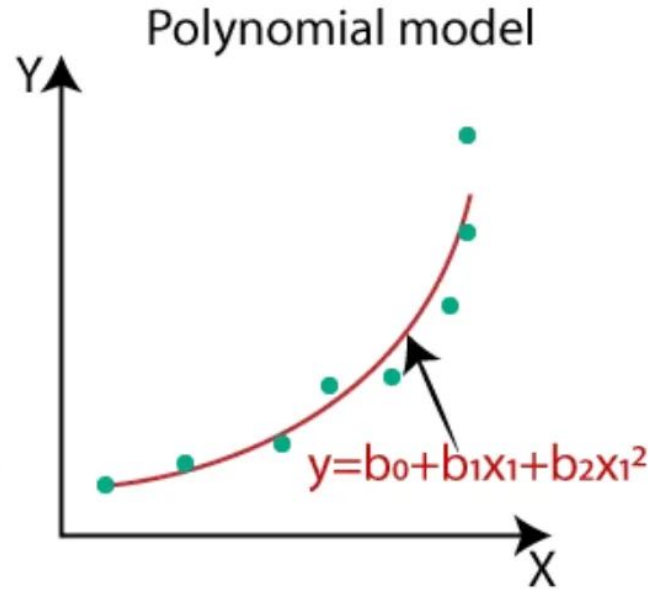
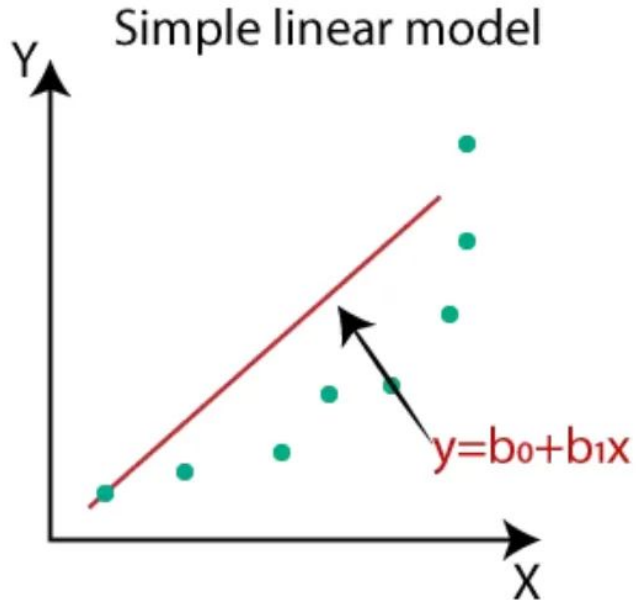
The resulting linear regression model would be:

$$y \approx 1.75x + 0.25$$

$$y \approx 1.75x + 0.25$$

Polynomial Regression

If the relationship between dependent and independent variables is not linear then linear regression cannot be used as it will draw large errors



Introduction

Polynomial regression models nonlinear relationships between a predictor variable x and a response variable y by fitting a polynomial equation to the data.

While it captures curvature in data, it remains a linear regression technique because the coefficients are linear combinations of the data

Introduction

A k-degree polynomial regression equation is:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_k x^k + e$$

Here, $\beta_0, \beta_1, \dots, \beta_k$ are coefficients, and e is the error term

The problem of non-linear regression can be solved using :

- a) Transformation of non-linear to linear so that linear regression can handle the data
- b) Use polynomial regression

Non linear transformation trick

Lets take an example of $y = ae^{bx}$. The transformation can be done by taking log on both sides

$$\ln(y) = \ln(a) + \ln(e^{bx})$$

$$\ln(y) = \ln(a) + bx * \ln(e)$$

$$\ln(y) = \ln(a) + bx$$

Polynomial regression

It can handle non linear relationships among variables

Higher-degree polynomials can fit noise rather than the underlying relationship. For example, a 5th-degree polynomial might overfit small datasets

Use metrics like R-squared or root mean squared error (RMSE) to compare polynomial degrees.

Consider the second degree of polynomial

$$y = a_0 + a_1x + a_2x^2$$

The coefficients of a_0, a_1 and a_2 are calculated using $a = X^{-1}B$

where

$$X = \begin{bmatrix} n & \sum x_i & \sum x_i^2 \\ \sum x_i & \sum x_i^2 & \sum x_i^3 \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 \end{bmatrix} \quad B = \begin{bmatrix} \sum y_i \\ \sum (x_i, y_i) \\ \sum (x_i^2, y_i) \end{bmatrix}$$

Find the polynomial regression equation for the following data

X	Y
1	1
2	4
3	9
4	15

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Example data
```

```
x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
```

```
y = np.array([1, 6, 17, 34, 57, 86, 121, 162, 209, 262, 321])
```

```
# Fit a degree-2 polynomial
```

```
degree = 2
```

```
coeffs = np.polyfit(x, y, degree)
```

```
# Generate polynomial function
```

```
poly_func = np.poly1d(coeffs)
```

```
# Predict y values
```

```
y_pred = poly_func(x)
```

```
# Print polynomial coefficients
```

```
print("Polynomial coefficients:", coeffs)
```

```
print("Polynomial equation: y = {:.1f}x^2 + {:.1f}x +  
{:.1f}".format(coeffs[0], coeffs[1], coeffs[2]))
```

```
# Plot
```

```
plt.scatter(x, y, label='Data')
```

```
plt.plot(x, y_pred, color='red', label='Fitted polynomial')
```

```
plt.legend()
```

```
plt.show()
```

Regularization in ML

While developing machine learning models, sometimes the training accuracy is high where as the testing accuracy or validation accuracy is low. This case is known as overfitting

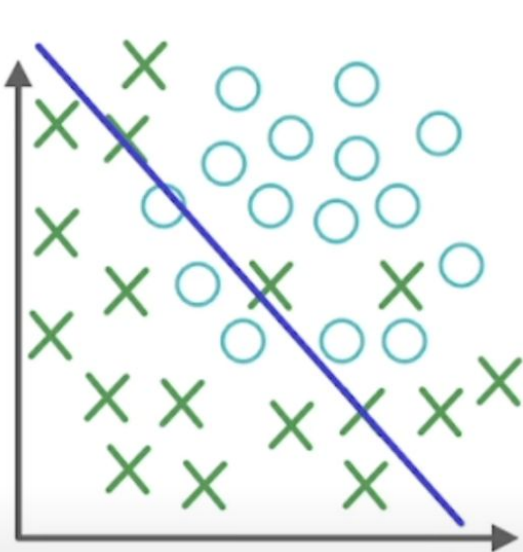
Overfitting is a phenomenon that occurs when a machine learning model is constrained to the training data and does not perform well on the unseen data. This is when model learns the noise instead of the pattern

Regularization in ML

Underfitting is the scenario where the model doesnot even perform on training data and hence it cannot perform on testing data either.

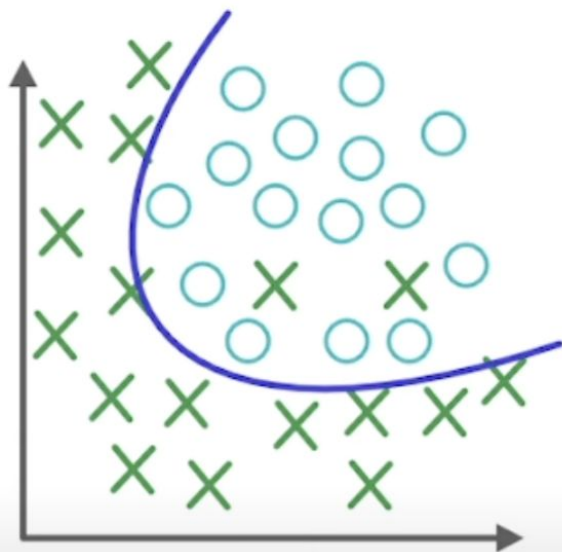
For this case we have to increase the complexity of the model or add some more features to the featured set

Regularization in Machine Learning

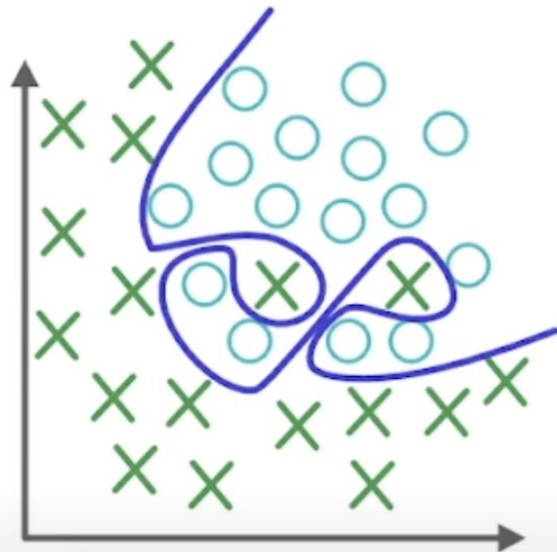


Under-fitting

(too simple to
explain the variance)



Appropriate-fitting



Over-fitting

(forcefitting--too
good to be true)

Bias and Variance

Bias refers to the errors which occur when we try to fit a statistical model on real world data which doesnot fit on some mathematical model.

If we use a simplistic model to fit the data, we can see high bias which refers to the case when the model is unable to learn the patterns in the data and performs poorly.

Variance implies the error value that occurs when we try to make predictions by using the data that is not previously not seen by the model

High variance occurs when the model learns noise present in the data

Regularization

It is a technique used to reduce errors by fitting the function appropriately on the given training set and avoid overfitting.

The commonly used regularization techniques are:

- a) Lasso Regularization - L1 Regularization
- b) Ridge Regularization - L2 Regularization
- c) Elastic Net Regularization - L1 and L2 Regularization

Lasso Regression

A regression model which uses the L1 Regularization Technique is called LASSO Regression. (Least Absolute Shrinkage and Selection Operator) regression.

Lasso Regression adds the “absolute value of magnitude” of the coefficient as the penalty term to the loss function (L)

Lasso Regression also helps us achieve feature selection by penalizing the weights approximately equal to zero if that feature does not serve any purpose in the model

Lasso Regression

$$Cost = \frac{1}{n} \sum_{i=1}^n (y_i - y_{pred})^2 + \lambda \sum_{i=1}^m |w_i|$$

where, m = no. of features

n = no. of examples

y_i = actual values

y_pred = predicted values

λ = hyperparameter that controls the strength of the penalty applied to the model's coefficients.

w_i = model coefficients

Ridge Regression

A regression model that uses L2 regularization technique is called ridge regression.

Ridge regression adds the “squared magnitude” of the coefficient as a penalty term to the loss function (L)

$$Cost = \frac{1}{n} \sum_{i=1}^n (y_i - y_{pred})^2 + \lambda \sum_{i=1}^m w_i^2$$

Elastic Net Regression

It uses both L1 and L2 regularization.

This implies that we add absolute number of the weights as well as the squared measure of the weights

$$Cost = \frac{1}{n} \sum_{i=1}^n (y_i - y_{pred})^2 + \lambda((1 - \alpha) \sum_{i=1}^m |w_i| + \alpha \sum_{i=1}^m w_i^2)$$

As λ increases, more coefficients are driven to exactly zero, eliminating less important features. This sparsity makes L1 effective for feature selection.

Bias-Variance Tradeoff

High λ (e.g., 10): Increases bias (simpler model) but reduces variance. Risk of underfitting.

Low λ (e.g., 0.01): Reduces bias (complex model) but increases variance. Risk of overfitting.

$\lambda=0$: Equivalent to ordinary least squares (no regularization)

OLS Linear Regression

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import numpy as np
```

```
X = np.array([[1], [2], [3], [4], [5], [6], [7], [8], [9], [10]])
```

```
y = np.array([3, 4, 2, 5, 6, 7, 8, 9, 10, 12])
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

OLS Linear Regression

```
model = LinearRegression()  
model.fit(X_train, y_train)  
y_pred = model.predict(X_test)  
mse = mean_squared_error(y_test, y_pred)  
print("OLS Linear Regression")  
print("Coefficients:", model.coef_)  
print("Intercept:", model.intercept_)  
print("Mean Squared Error:", mse)
```

L2 regression

```
from sklearn.linear_model import Ridge  
  
# Initialize Ridge with alpha = 1.0 (regularization strength)  
ridge_model = Ridge(alpha=1.0)  
ridge_model.fit(X_train, y_train)  
  
y_pred_ridge = ridge_model.predict(X_test)  
mse_ridge = mean_squared_error(y_test, y_pred_ridge)  
  
print("\nRidge Regression")  
print("Coefficients:", ridge_model.coef_)  
print("Intercept:", ridge_model.intercept_)  
print("Mean Squared Error:", mse_ridge)
```

L1 Regularization

```
from sklearn.linear_model import Lasso

# Initialize Lasso with alpha = 0.1
lasso_model = Lasso(alpha=0.1)

lasso_model.fit(X_train, y_train)

y_pred_lasso = lasso_model.predict(X_test)

mse_lasso = mean_squared_error(y_test, y_pred_lasso)

print("\nLasso Regression")

print("Coefficients:", lasso_model.coef_)

print("Intercept:", lasso_model.intercept_)

print("Mean Squared Error:", mse_lasso)
```

Decision Tree

A decision tree is a machine learning algorithm used for classification and regression tasks.

It works by recursively partitioning the input data based on the values of its features until a stopping criterion is reached.

Introduction

- A classification scheme which generates a tree and a set of rules from given data set.
- The set of records available for developing classification methods is divided into two disjoint subsets – a *training set* and a *test set*.
- The attributes of the records are categorise into two types:
 - Attributes whose domain is numerical are called **numerical attributes**.
 - Attributes whose domain is not numerical are called the **categorical attributes**.

Introduction

- A *decision tree* is a tree with the following properties:
 - An **inner node** represents an **attribute**.
 - An **edge** represents a **test** on the attribute of the father node.
 - A **leaf** represents one of the **classes**.
- Construction of a decision tree
 - Based on the training data
 - Top-Down strategy

Decision Tree

Example

Table 6.1 Training Data Set

OUTLOOK	TEMP(F)	HUMIDITY(%)	WINDY	CLASS
sunny	79	90	true	no play
sunny	56	70	false	play
sunny	79	75	true	play
sunny	60	90	true	no play
overcast	88	88	false	no play
overcast	63	75	true	play
overcast	88	95	false	play
rain	78	60	false	play
rain	66	70	false	no play
rain	68	60	true	no play

- The data set has five attributes.
- There is a special attribute: the attribute *class* is the class label.
- The attributes, *temp* (temperature) and *humidity* are numerical attributes
- Other attributes are categorical, that is, they cannot be ordered.
- Based on the training data set, we want to find a set of rules to know what values of *outlook*, *temperature*, *humidity* and *wind*, determine whether or not to play golf.

Decision Tree

Example

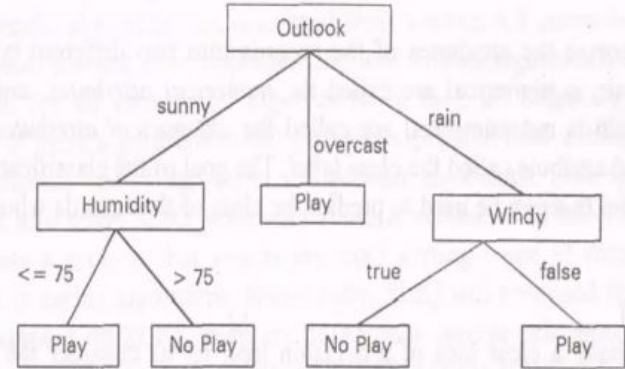
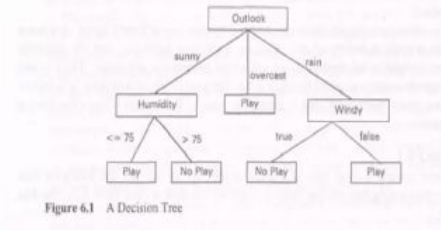


Figure 6.1 A Decision Tree

- We have five leaf nodes.
- In a decision tree, each leaf node represents a rule.
- We have the following rules corresponding to the tree given in Figure.
 - **RULE 1** *If it is sunny and the humidity is not above 75%, then play.*
 - **RULE 2** *If it is sunny and the humidity is above 75%, then do not play.*
 - **RULE 3** *If it is overcast, then play.*
 - **RULE 4** *If it is rainy and not windy, then play.*
 - **RULE 5** *If it is rainy and windy, then don't play.*

Classification



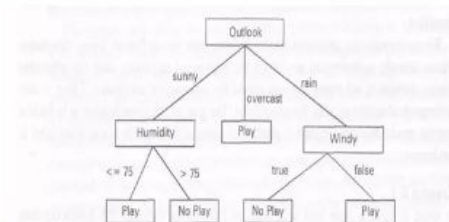
- The classification of an unknown input vector is done by traversing the tree from the root node to a leaf node.
- A record enters the tree at the root node.
- At the root, a test is applied to determine which child node the record will encounter next.
- This process is repeated until the record arrives at a leaf node.
- All the records that end up at a given leaf of the tree are classified in the same way.
- There is a unique path from the root to each leaf.
- The path is a rule which is used to classify the records.



Figure 6.1 A Decision Tree

- In our tree, we can carry out the classification for an unknown record as follows.
- Let us assume, for the record, that we know the values of the first four attributes (but we do not know the value of *class* attribute) as
- *outlook= rain; temp = 70; humidity = 65; and windy= true.*

- We start from the root node to check the value of the attribute associated at the root node.
- This attribute is the *splitting attribute* at this node.
- For a decision tree, at every node there is an attribute associated with the node called the *splitting attribute*.
- In our example, *outlook* is the splitting attribute at root.
- Since for the given record, *outlook* = *rain*, we move to the right-most child node of the root.
- At this node, the splitting attribute is *windy* and we find that for the record we want classify, *windy* = *true*.
- Hence, we move to the left child node to conclude that the class label is "*no play*".



Advantages and Shortcomings of Decision Tree Classifications

- A decision tree construction process is concerned with identifying the splitting attributes and splitting criterion at every level of the tree.
- Major strengths are:
 - Decision tree able to generate understandable rules.
 - They are able to handle both numerical and categorical attributes.
 - They provide clear indication of which fields are most important for prediction or classification.
- Weaknesses are:
 - The process of growing a decision tree is computationally expensive. At each node, each candidate splitting field is examined before its best split can be found.
 - Some decision tree can only deal with binary-valued target classes.

Steps in Decision Tree

1. Data preparation: The first step is to prepare the input data in a format suitable for the algorithm. This involves selecting the relevant features, cleaning and normalizing the data.
2. Choosing the best feature: The algorithm then selects the feature that provides the most information gain or the best split in the data. The information gain is calculated by comparing the impurity of the data before and after the split.
3. Creating the tree: The algorithm then creates a decision node at the top of the tree using the selected feature. It then creates child nodes for each possible value of the feature.

Steps in Decision Tree

4. Recursive partitioning: The algorithm then recursively applies the same process to each child node, selecting the best feature and creating child nodes until a stopping criterion is reached. The stopping criterion can be based on the depth of the tree, the number of data points in a node or other factors.
5. Making predictions: Once the tree is created, it can be used to make predictions on new data by traversing the tree from the root node to a leaf node. At each decision node, the value of the corresponding feature in the new data is used to decide which child node to follow. The prediction is then made based on the majority class or the mean value of the training data in the leaf node.

Some of the examples area of decision tree

1. Predicting whether a customer will buy a product based on their age, income, and occupation.
2. Predicting the likelihood of a loan default based on the borrower's credit score, income, and debt-to-income ratio.
3. Classifying emails as spam or non-spam based on features such as the sender, subject line, and body text.
4. Predicting the price of a house based on its location, size, and number of bedrooms and bathrooms.
5. Diagnosing medical conditions based on symptoms and patient characteristics.
6. Identifying fraudulent transactions in financial data based on transaction features such as amount, time, and location.

Some key terminologies commonly used in decision trees

- Root node: The top-most node of a decision tree which represents the entire population or sample being analyzed.
- Internal nodes: The intermediate nodes of a decision tree that represent a split on a specific feature and separate the dataset into smaller subsets.
- Leaf nodes: The bottom-most nodes of a decision tree that represent the final outcomes or class labels of the analysis.
- Branches: The lines connecting nodes that represent a decision or a split based on a feature.

Some key terminologies commonly used in decision trees

- Splitting criterion: The measure used to determine the quality of a split or decision, such as entropy, information gain, or Gini index.
- Pruning: The process of removing branches or nodes from a decision tree to improve its accuracy or simplicity.
- Decision boundary: The boundary that separates the different classes or outcomes in the decision tree.
- Feature importance: The measure of the relative importance of each feature in making a decision or prediction in the decision tree.

Other terms in Decision Tree

1. Entropy: Entropy is the measure of uncertainty or randomness in a data set. Entropy handles how a decision tree splits the data.

It is calculated using the following formula:

$$\sum_{i=1}^k P(value_i) \cdot \log_2(P(value_i))$$

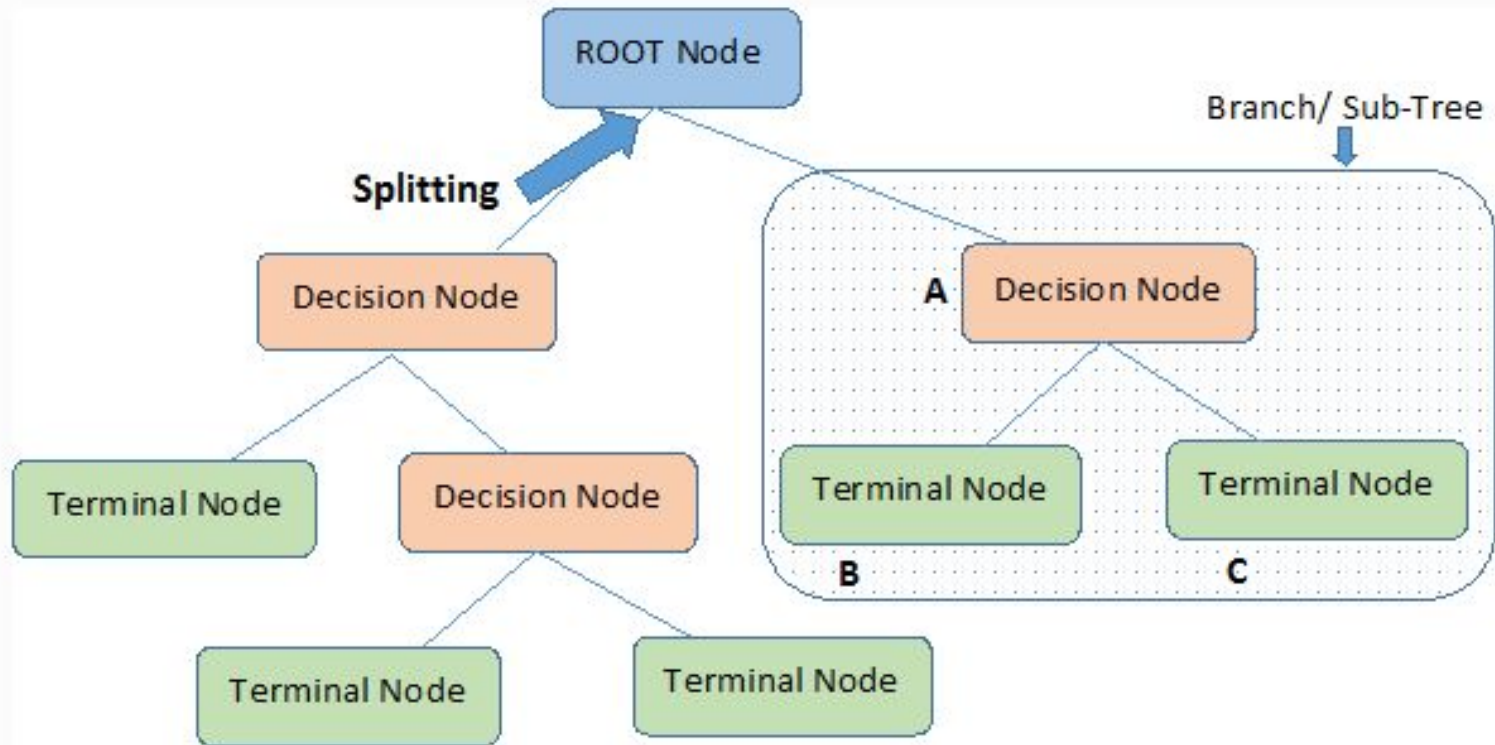
2. Information Gain: The information gain measures the decrease in entropy after the data set is split.

It is calculated as follows:

$$IG(Y, X) = \text{Entropy}(Y) - \text{Entropy}(Y | X)$$

3. Gini Index: The Gini Index is used to determine the correct variable for splitting nodes. It measures how often a randomly chosen variable would be incorrectly identified.

Some key terminologies commonly used in decision trees



Note:- A is parent node of B and C.

ID3 Algorithm

- Input the training data set: The ID3 algorithm takes a set of input data that is used to build a decision tree. The input data should have several attributes or features, and each attribute should have a set of possible values.
- Calculate the entropy of the target variable: The entropy of the target variable is a measure of the amount of uncertainty in the data. The ID3 algorithm calculates the entropy of the target variable using the formula:
$$\text{entropy}(S) = -p_1 \log_2(p_1) - p_2 \log_2(p_2) - \dots - p_n \log_2(p_n)$$
where S is the set of training data, and $p_1, p_2, \dots, p_n$ are the probabilities of each possible outcome of the target variable.

ID3 Algorithm

- Calculate the information gain for each attribute: The information gain of an attribute is a measure of how well it separates the data into different classes. The ID3 algorithm calculates the information gain of each attribute using the formula:
$$\text{Information_gain}(S, A) = \text{entropy}(S) - \sum (|S_v|/|S|) * \text{entropy}(S_v)$$
where A is an attribute, S_v is the subset of S for which A has the value v , and $|S|$ and $|S_v|$ are the number of elements in S and S_v , respectively.
- Select the attribute with the highest information gain: The ID3 algorithm selects the attribute with the highest information gain as the root node of the decision tree.
- Create a branch for each possible value of the selected attribute: The ID3 algorithm creates a branch for each possible value of the selected attribute and partitions the data accordingly.

ID3 Algorithm

- Repeat the process for each branch: The ID3 algorithm recursively repeats the process for each branch until all the data is classified.
- Prune the decision tree: The ID3 algorithm may overfit the training data, resulting in a decision tree that is too complex and not generalizable to new data. To avoid this problem, the ID3 algorithm may prune the decision tree by removing branches that do not improve the accuracy of the model.
- Use the decision tree to classify new data: Once the decision tree is constructed, it can be used to classify new data by traversing the tree and assigning a class label to the leaf node that is reached.

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Outlook

Values (Outlook) = Sunny, Overcast, Rain

$$S = [9+, 5-]$$

$$Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{Sunny} \leftarrow [2+, 3-]$$

$$Entropy(S_{Sunny}) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.971$$

$$S_{Overcast} \leftarrow [4+, 0-]$$

$$Entropy(S_{Overcast}) = -\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4} = 0$$

$$S_{Rain} \leftarrow [3+, 2-]$$

$$Entropy(S_{Rain}) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.971$$

$$Gain(S, Outlook) = Entropy(S) - \sum_{v \in \{Sunny, Overcast, Rain\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Outlook)$$

$$= Entropy(S) - \frac{5}{14} Entropy(S_{Sunny}) - \frac{4}{14} Entropy(S_{Overcast}) - \frac{5}{14} Entropy(S_{Rain})$$

$$Gain(S, Outlook) = 0.94 - (5/14)*0.971 - (4/14)*0 - (5/14)*0.971 = 0.2464$$

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Temp

Values (Temp) = Hot, Mild, Cool

$$S = [9+, 5-]$$

$$Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{Hot} \leftarrow [2+, 2-]$$

$$Entropy(S_{Hot}) = -\frac{2}{4} \log_2 \frac{2}{4} - \frac{2}{4} \log_2 \frac{2}{4} = 1.0$$

$$S_{Mild} \leftarrow [4+, 2-]$$

$$Entropy(S_{Mild}) = -\frac{4}{6} \log_2 \frac{4}{6} - \frac{2}{6} \log_2 \frac{2}{6} = 0.9183$$

$$S_{Cool} \leftarrow [3+, 1-]$$

$$Entropy(S_{Cool}) = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} = 0.8113$$

$$Gain(S, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Temp)$$

$$= Entropy(S) - \frac{4}{14} Entropy(S_{Hot}) - \frac{6}{14} Entropy(S_{Mild})$$

$$- \frac{4}{14} Entropy(S_{Cool})$$

$$Gain(S, Temp) = 0.94 - (4/14)*1 - (6/14)*0.9183 - (4/14)*0.8113 = 0.0289$$

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Humidity

Values (Humidity) = High, Normal

$$S = [9+, 5 -]$$

$$Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{High} \leftarrow [3+, 4 -]$$

$$Entropy(S_{High}) = -\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7} = 0.9852$$

$$S_{Normal} \leftarrow [6+, 1 -]$$

$$Entropy(S_{Normal}) = -\frac{6}{7} \log_2 \frac{6}{7} - \frac{1}{7} \log_2 \frac{1}{7} = 0.5916$$

$$Gain(S, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Humidity)$$

$$= Entropy(S) - \frac{7}{14} Entropy(S_{High}) - \frac{7}{14} Entropy(S_{Normal})$$

$$Gain(S, Humidity) = 0.94 - (7/14) * 0.9852 - (7/14) * 0.5916 = 0.1516$$

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Wind

Values (Wind) = Strong, Weak

$$S = [9+, 5-]$$

$$Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{Strong} \leftarrow [3+, 3-]$$

$$Entropy(S_{Strong}) = 1.0$$

$$S_{Weak} \leftarrow [6+, 2-]$$

$$Entropy(S_{Weak}) = -\frac{6}{8} \log_2 \frac{6}{8} - \frac{2}{8} \log_2 \frac{2}{8} = 0.8113$$

$$Gain(S, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Wind) = Entropy(S) - \frac{6}{14} Entropy(S_{Strong}) - \frac{8}{14} Entropy(S_{Weak})$$

$$Gain(S, Wind) = Entropy(S) - (6/14)*1 - (8/14)*0.8113 = 0.0478$$

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

$$Gain(S, Outlook) = 0.2464$$

$$Gain(S, Temp) = 0.0289$$

$$Gain(S, Humidity) = 0.1516$$

$$Gain(S, Wind) = 0.0478$$

$\{D1, D2, \dots, D14\}$

$[9+, 5-]$

Outlook

Sunny

Overcast

Rain

$\{D1, D2, D8, D9, D11\}$

$\{D3, D7, D12, D13\}$

$\{D4, D5, D6, D10, D14\}$

$[2+, 3-]$

$[4+, 0-]$

$[3+, 2-]$

?

Yes

?

Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Attribute: Temp

Values (Temp) = Hot, Mild, Cool

$$S_{Sunny} = [2+, 3-]$$

$$Entropy(S_{Sunny}) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.97$$

$$S_{Hot} \leftarrow [0+, 2-]$$

$$Entropy(S_{Hot}) = 0.0$$

$$S_{Mild} \leftarrow [1+, 1-]$$

$$Entropy(S_{Mild}) = 1.0$$

$$S_{Cool} \leftarrow [1+, 0-]$$

$$Entropy(S_{Cool}) = 0.0$$

$$Gain(S_{Sunny}, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Sunny}, Temp)$$

$$= Entropy(S) - \frac{2}{5} Entropy(S_{Hot}) - \frac{2}{5} Entropy(S_{Mild})$$

$$- \frac{1}{5} Entropy(S_{Cool})$$

$$Gain(S_{sunny}, Temp) = 0.97 - (\frac{2}{5}) * 0 - (\frac{2}{5}) * 1 - (\frac{1}{5}) * 0 = 0.570$$

Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Attribute: Humidity

Values (Humidity) = High, Normal

$$S_{Sunny} = [2+, 3-]$$

$$Entropy(S) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.97$$

$$S_{high} \leftarrow [0+, 3-]$$

$$Entropy(S_{High}) = 0.0$$

$$S_{Normal} \leftarrow [2+, 0-]$$

$$Entropy(S_{Normal}) = 0.0$$

$$Gain(S_{Sunny}, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Sunny}, Humidity) = Entropy(S) - \frac{3}{5} Entropy(S_{High}) - \frac{2}{5} Entropy(S_{Normal})$$

$$Gain(S_{sunny}, Humidity) = 0.97 - (3/5) * 0 - (2/5) * 0 = 0.97$$

Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Attribute: Wind

Values (Wind) = Strong, Weak

$$S_{Sunny} = [2+, 3-]$$

$$Entropy(S) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.97$$

$$S_{Strong} \leftarrow [1+, 1-]$$

$$Entropy(S_{Strong}) = 1.0$$

$$S_{Weak} \leftarrow [1+, 2-]$$

$$Entropy(S_{Weak}) = -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} = 0.9183$$

$$Gain(S_{Sunny}, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Sunny}, Wind) = Entropy(S) - \frac{2}{5} Entropy(S_{Strong}) - \frac{3}{5} Entropy(S_{Weak})$$

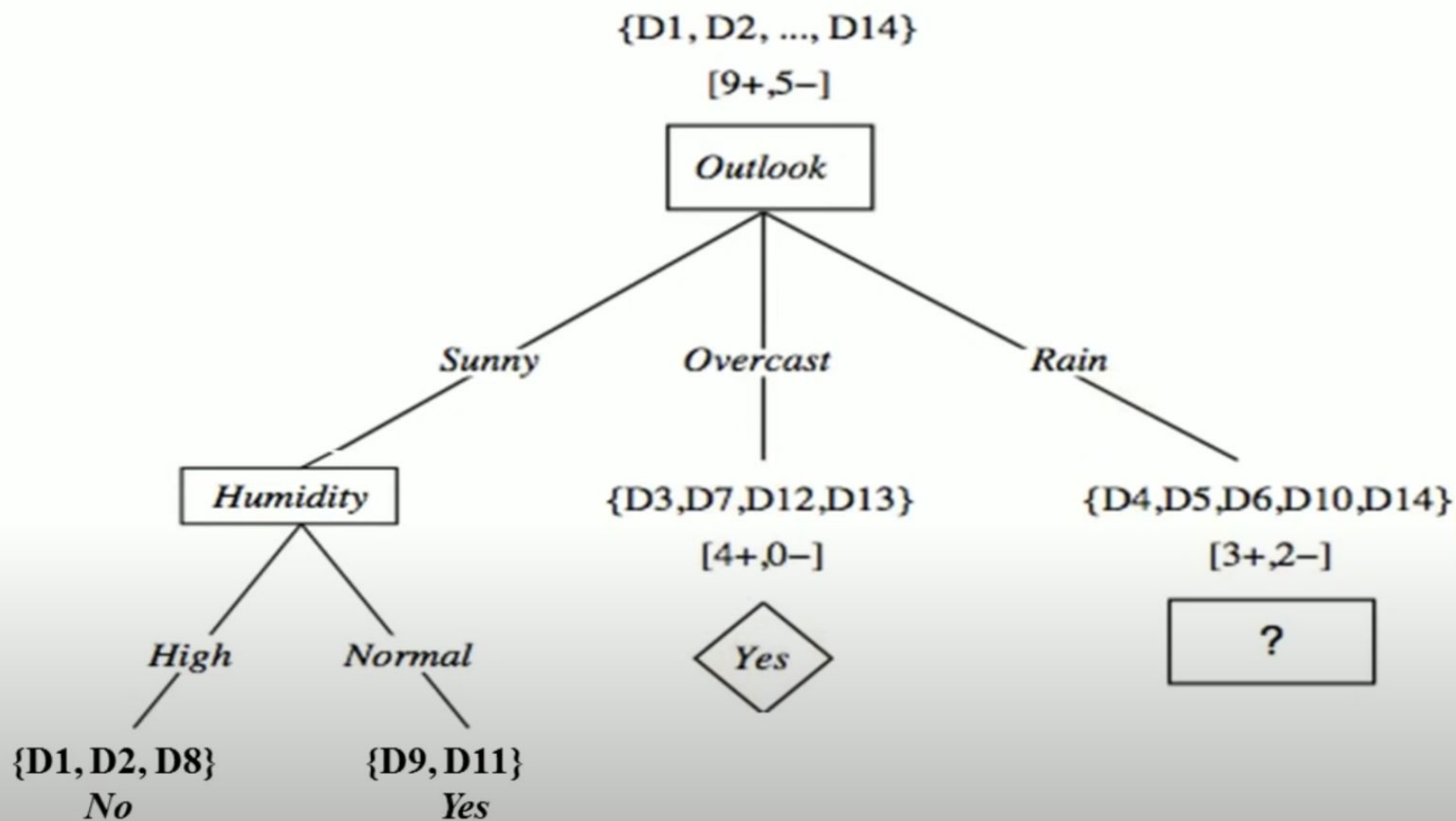
$$Gain(S_{Sunny}, Wind) = 0.97 - (\frac{2}{5}) * 1 - (\frac{3}{5}) * 0.9183 = 0.0192$$

Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

$$Gain(S_{sunny}, Temp) = 0.570$$

$$Gain(S_{sunny}, Humidity) = 0.97$$

$$Gain(S_{sunny}, Wind) = 0.0192$$



Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Attribute: Temp

Values (Temp) = Hot, Mild, Cool

$$S_{Rain} = [3+, 2-]$$

$$Entropy(S_{Sunny}) = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} = 0.97$$

$$S_{Hot} \leftarrow [0+, 0-]$$

$$Entropy(S_{Hot}) = 0.0$$

$$S_{Mild} \leftarrow [2+, 1-]$$

$$Entropy(S_{Mild}) = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = 0.9183$$

$$S_{Cool} \leftarrow [1+, 1-]$$

$$Entropy(S_{Cool}) = 1.0$$

$$Gain(S_{Rain}, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Rain}, Temp)$$

$$= Entropy(S) - \frac{0}{5} Entropy(S_{Hot}) - \frac{3}{5} Entropy(S_{Mild})$$

$$- \frac{2}{5} Entropy(S_{Cool})$$

$$Gain(S_{RAIN}, Temp) = 0.97 - (0/5)*0 - (3/5)*0.9183 - (2/5)*1 = 0.0192$$

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Attribute: Humidity

Values (Humidity) = High, Normal

$$S_{Rain} = [3+, 2-]$$

$$Entropy(S_{Sunny}) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.97$$

$$S_{High} \leftarrow [1+, 1-]$$

$$Entropy(S_{High}) = 1.0$$

$$S_{Normal} \leftarrow [2+, 1-]$$

$$Entropy(S_{Normal}) = -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} = 0.9183$$

$$Gain(S_{Rain}, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Rain}, Humidity) = Entropy(S) - \frac{2}{5} Entropy(S_{High}) - \frac{3}{5} Entropy(S_{Normal})$$

$$Gain(S_{RAIN}, Humidity) = 0.97 - (\frac{2}{5}) * 1 - (\frac{3}{5}) * 0.9183 = 0.0192$$

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Attribute: Wind

Values (wind) = Strong, Weak

$$S_{Rain} = [3+, 2-]$$

$$Entropy(S_{Sunny}) = -\frac{3}{5}\log_2 \frac{3}{5} - \frac{2}{5}\log_2 \frac{2}{5} = 0.97$$

$$S_{Strong} \leftarrow [0+, 2-]$$

$$Entropy(S_{Strong}) = 0.0$$

$$S_{Weak} \leftarrow [3+, 0-]$$

$$Entropy(S_{Weak}) = 0.0$$

$$Gain(S_{Rain}, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Rain}, Wind) = Entropy(S) - \frac{2}{5} Entropy(S_{Strong}) - \frac{3}{5} Entropy(S_{Weak})$$

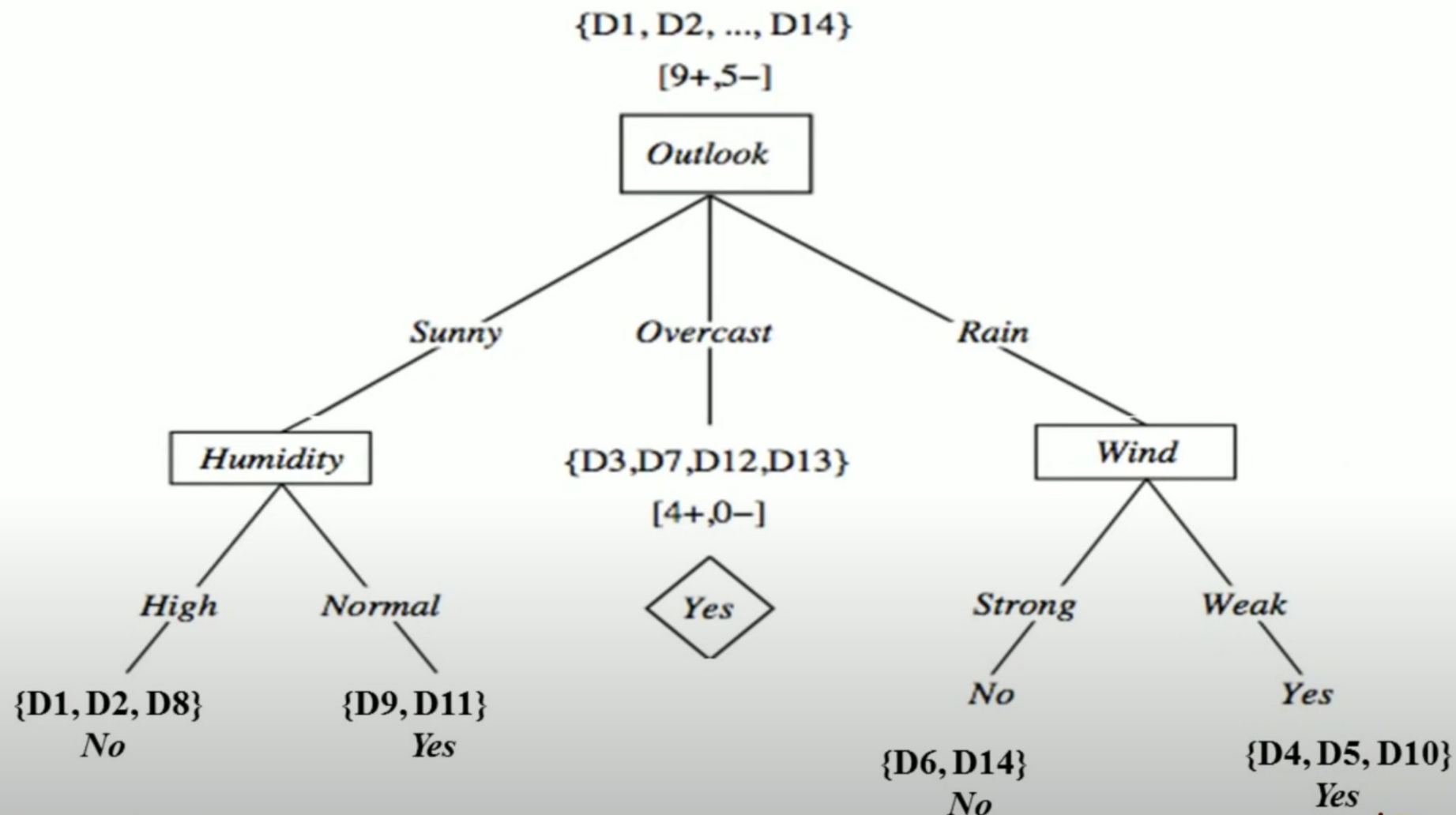
$$Gain(S_{Rain}, Wind) = 0.97 - \left(\frac{2}{5}\right) * 0 - \left(\frac{3}{5}\right) * 0 = 0.97$$

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

$$Gain(S_{Rain}, Temp) = 0.0192$$

$$Gain(S_{Rain}, Humidity) = 0.0192$$

$$Gain(S_{Rain}, Wind) = 0.97$$



Random Forest Classifier

The decision tree concept is more to the rule-based system. Given the training dataset with targets and features, the decision tree algorithm will come up with some set of rules.

The same set of rules can be used to perform the prediction on the test dataset.

In a decision tree algorithm, metrics such as information gain and Gini index are calculated to evaluate the quality of splits.

Based on these calculations, the algorithm identifies decision rules that serve as criteria to split the features into multiple nodes, thereby partitioning the data to increase purity in each node.

In a random forest algorithm, Instead of using information gain or Gini index for calculating the root node, the process of finding the root node and splitting the feature nodes will happen randomly.

Random Forest Classifier

Decision trees having low bias and high variance tends to overfit the data.

So bagging technique becomes a very good solution for decreasing the variance in a decision tree.

Instead of using a bagging model with an underlying model as a decision tree, we can also use Random forest which is more convenient and well optimized for decision trees.

Random Forest Classifier

Random forest is a supervised learning algorithm that is used for both classification as well as regression.

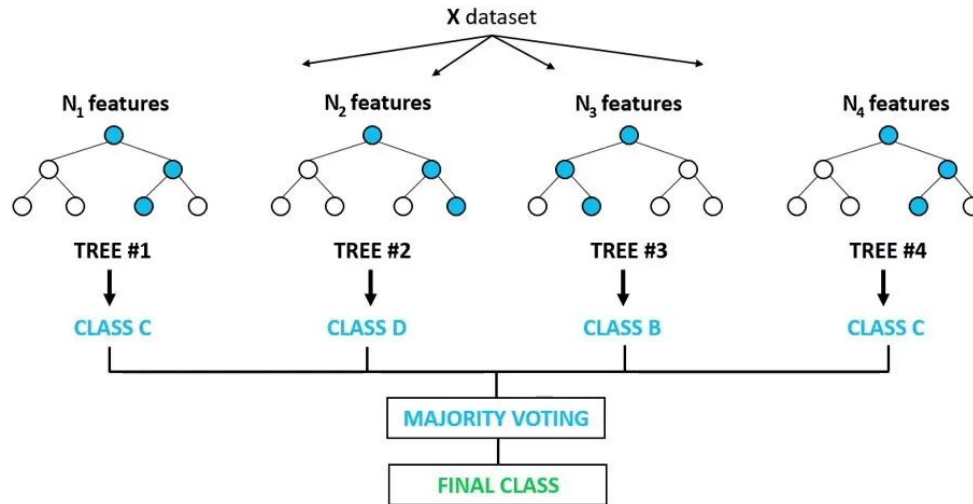
However, it is mainly used for classification problems.

As we know that a forest is made up of trees and more trees means more robust forest.

Similarly, a random forest algorithm creates decision trees on data samples and then gets the prediction from each of them and finally selects the best solution.

Random Forest Classifier

Random Forest Classifier



Random Forest Classifier

Decision trees	Random Forest
1. Decision trees normally suffer from the problem of overfitting if it's allowed to grow without any control.	1. Random forests are created from subsets of data, and the final output is based on average or majority ranking; hence the problem of overfitting is taken care of.
2. A single decision tree is faster in computation.	2. It is comparatively slower.
3. When a data set with features is taken as input by a decision tree, it will formulate some rules to make predictions.	3. Random forest randomly selects observations, builds a decision tree, and takes the average result. It doesn't use any set of formulas.

Advantages of Random Forest Classifier

It overcomes the problem of overfitting by averaging or combining the results of different decision trees.

Random forests work well for a large range of data items than a single decision tree does.

Random forest has less variance than a single decision tree.

Random forests are very flexible and possess very high accuracy.

Scaling of data is not required in a random forest algorithm. It maintains good accuracy even after providing data without scaling.

Random Forest algorithms maintain good accuracy even when a large proportion of the data is missing.

Disadvantages of Random Forest Classifier

Complexity is the main disadvantage of Random forest algorithms.

Construction of Random forests are much harder and time-consuming than decision trees.

More computational resources are required to implement the Random Forest algorithm.

It is less intuitive in the case when we have a large collection of decision trees.

The prediction process using random forests is very time-consuming in comparison with other algorithms.

SVM

Support vector machines are a set of supervised learning methods used for classification, regression, and outliers detection. All of these are common tasks in machine learning.

You can use them to detect cancerous cells based on millions of images or you can use them to predict future driving routes with a well-fitted regression model.

There are specific types of SVMs you can use for particular machine learning problems, like support vector regression (SVR) which is an extension of support vector classification (SVC).

The main thing to keep in mind here is that these are just math equations tuned to give you the most accurate answer possible as quickly as possible.

SVM

SVMs are different from other classification algorithms because of the way they choose the decision boundary that maximizes the distance from the nearest data points of all the classes.

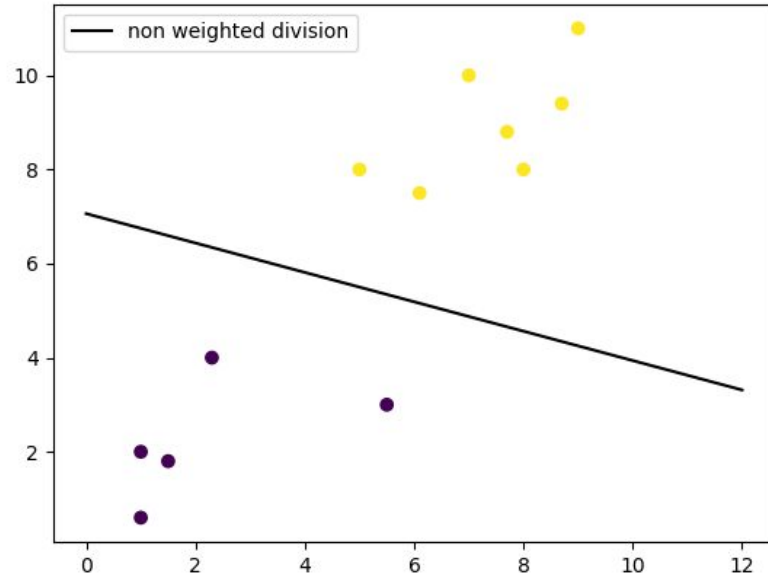
The decision boundary created by SVMs is called the maximum margin classifier or the maximum margin hyper plane.

How SVM works?

A simple linear SVM classifier works by making a straight line between two classes. That means all of the data points on one side of the line will represent a category and the data points on the other side of the line will be put into a different category. This means there can be an infinite number of lines to choose from.

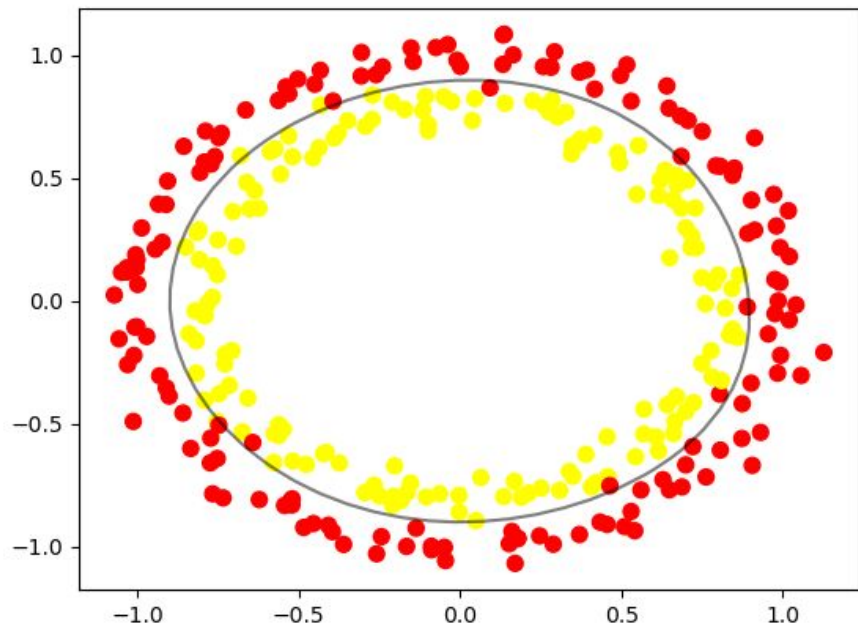
What makes the linear SVM algorithm better than some of the other algorithms, like k-nearest neighbors, is that it chooses the best line to classify your data points. It chooses the line that separates the data and is the furthest away from the closest data points as possible.

The two closest data points give you the support vectors you'll use to find that line. That line is called the decision bo



The decision boundary doesn't have to be a line.

It's also referred to as a hyperplane because you can find the decision boundary with any number of features, not just two.



Applications

SVMs are used in applications like handwriting recognition, intrusion detection, face detection, email classification, gene classification, and in web pages. This is one of the reasons we use SVMs in machine learning. It can handle both classification and regression on linear and non-linear data.

Another reason we use SVMs is because they can find complex relationships between your data without you needing to do a lot of transformations on your own. It's a great option when you are working with smaller datasets that have tens to hundreds of thousands of features. They typically find more accurate results when compared to other algorithms because of their ability to handle small, complex datasets.

Advantages of SVM

1. SVM works well in high-dimensional spaces, making it suitable for many real-world applications.
2. It is effective in cases where the number of dimensions is greater than the number of samples.
3. SVM provides a way to control overfitting, by maximizing the margin between the classes.
4. SVM can handle non-linear decision boundaries by using kernel functions.

Limitation of SVM

1. SVM may be computationally expensive, especially for large datasets.
2. SVM can be sensitive to the choice of kernel function and other parameters.
3. SVM may not perform well if the classes are highly overlapping or if there is a lot of noise in the data.

Types of SVM

- Simple SVM: Typically used for linear regression and classification problems.
- Kernel SVM: Has more flexibility for non-linear data because you can add more features to fit a hyperplane instead of a two-dimensional space.

Kernel Functions

Linear

These are commonly recommended for text classification because most of these types of classification problems are linearly separable.

The linear kernel works really well when there are a lot of features, and text classification problems have a lot of features. Linear kernel functions are faster than most of the others and you have fewer parameters to optimize.

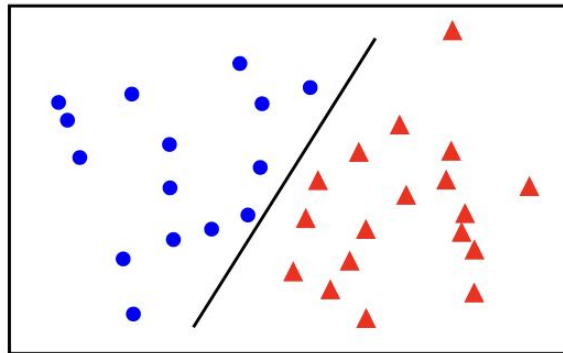
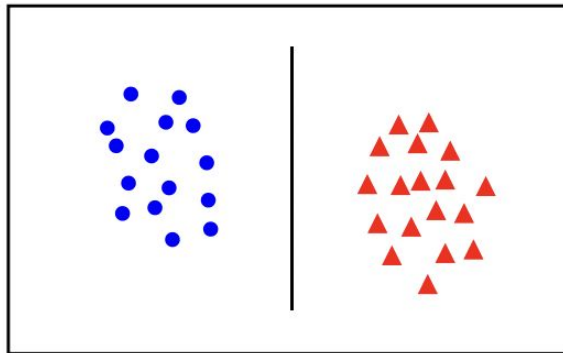
Here's the function that defines the linear kernel:

$$f(X) = w^T * X + b$$

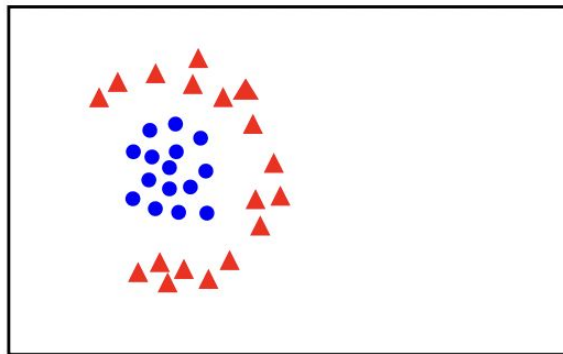
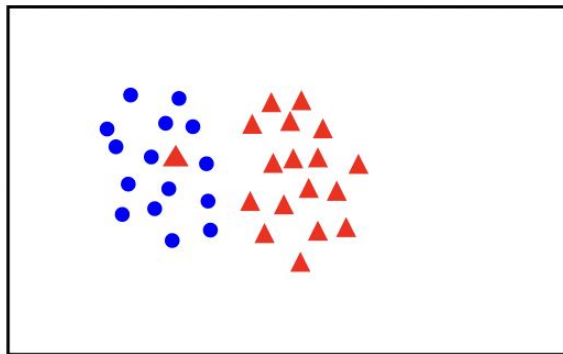
w is the weight vector that you want to minimize, **X** is the data that you're trying to classify, and **b** is the linear coefficient estimated from the training data. This equation defines the decision boundary that the SVM returns.

Linear separability

linearly
separable



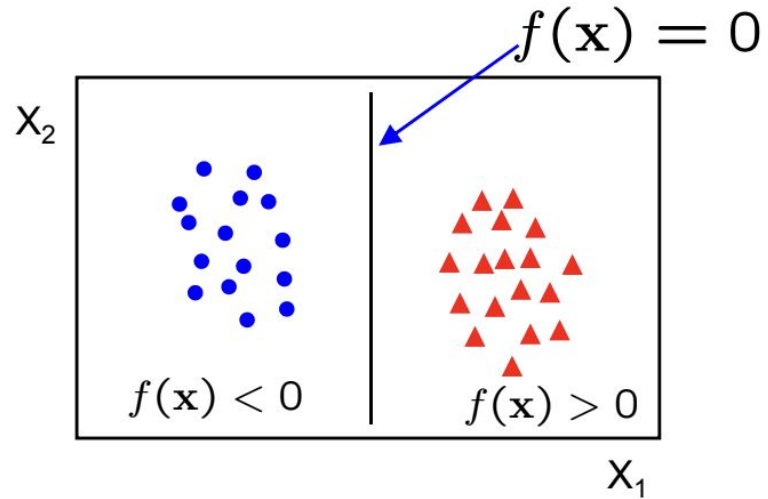
not
linearly
separable



Linear classifiers

A linear classifier has the form

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$

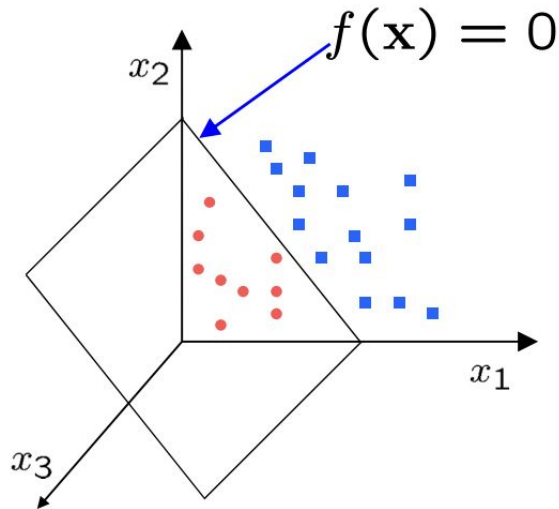


- in 2D the discriminant is a line
- $\mathbf{w}$ is the **normal** to the line, and b the **bias**
- $\mathbf{w}$ is known as the **weight vector**

Linear classifiers

A linear classifier has the form

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$



- in 3D the discriminant is a plane, and in nD it is a hyperplane

For a K-NN classifier it was necessary to 'carry' the training data

For a linear classifier, the training data is used to learn $\mathbf{w}$ and then discarded

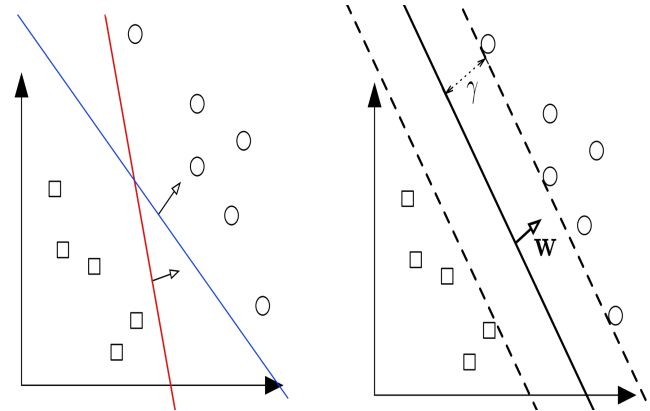
Only $\mathbf{w}$ is needed for classifying new data

Separating Data with Maximum Margins

The goal of SVMs is to find a hyperplane in a high-dimensional space that separates the data into two classes with maximum margin.

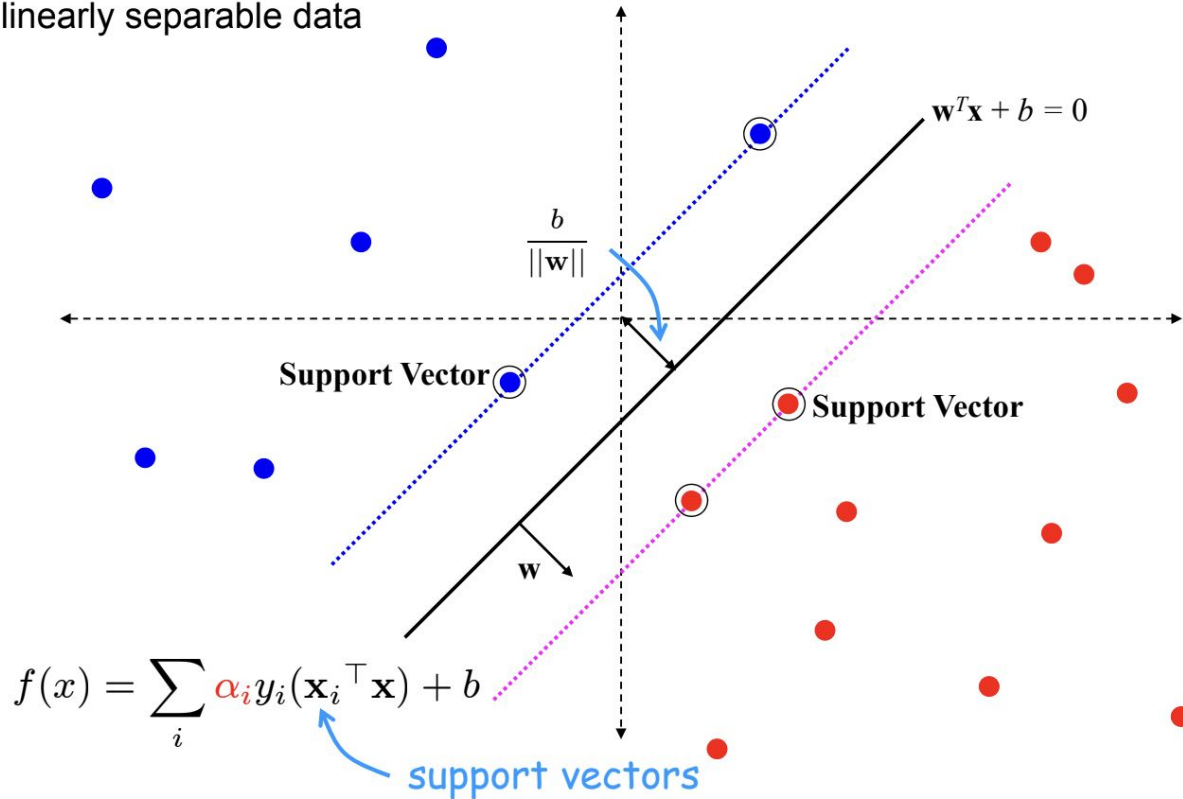
The margin is defined as the distance between the hyperplane and the closest data points from each class. The goal of SVM is to find a hyperplane with the maximum margin, which means that it separates the two classes and maximizes the distance between the closest points and the hyperplane.

To find the hyperplane with maximum margin, SVMs use a training dataset to learn the optimal parameters of the hyperplane. These parameters are learned by solving an optimization problem that involves maximizing the margin subject to certain constraints. These constraints ensure that all the data points are correctly classified and lie on the correct side of the hyperplane.



Support Vector Machine

linearly separable data



Support Vector Machine - Linear Example Solved

Suppose we are given the following positively labeled data points,

$$\left\{ \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \begin{pmatrix} 3 \\ -1 \end{pmatrix}, \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \begin{pmatrix} 6 \\ -1 \end{pmatrix} \right\}$$

and the following negatively labeled data points,

$$\left\{ \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ 0 \end{pmatrix} \right\}$$

From the graph drawn, we can see that there are three support vectors.

$$s1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, s2 = \begin{pmatrix} 3 \\ 1 \end{pmatrix}, s3 = \begin{pmatrix} 3 \\ -1 \end{pmatrix}$$

- Each vector is augmented with a 1 as a bias input

- So, $s_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, then $\tilde{s}_1 = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$

- Similarly,

- $s_2 = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$, then $\tilde{s}_2 = \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix}$ and $s_3 = \begin{pmatrix} 3 \\ -1 \end{pmatrix}$, then $\tilde{s}_3 = \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix}$

$$\alpha_1 \tilde{s}_1 \cdot \tilde{s}_1 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_1 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_1 = -1$$

$$\alpha_1 \tilde{s}_1 \cdot \tilde{s}_2 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_2 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_2 = +1$$

$$\alpha_1 \tilde{s}_1 \cdot \tilde{s}_3 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_3 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_3 = +1$$

$$\alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} = -1$$

$$\alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} = 1$$

$$\alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} = 1$$

$$\alpha_1(1 + 0 + 1) + \alpha_2(3 + 0 + 1) + \alpha_3(3 + 0 + 1) = -1$$

$$\alpha_1(3 + 0 + 1) + \alpha_2(9 + 1 + 1) + \alpha_3(9 - 1 + 1) = 1$$

$$\alpha_1(3 + 0 + 1) + \alpha_2(9 - 1 + 1) + \alpha_3(9 + 1 + 1) = 1$$

$$2\alpha_1 + 4\alpha_2 + 4\alpha_3 = -1$$

$$4\alpha_1 + 11\alpha_2 + 9\alpha_3 = 1$$

$$4\alpha_1 + 9\alpha_2 + 11\alpha_3 = 1$$

$$\alpha_1 = -3.5, \alpha_2 = \alpha_3 = 0.75$$

Support Vector Machine - Linear Example Solved

$$\begin{aligned}\tilde{w} &= \sum_i \alpha_i \tilde{s}_i \\ &= -3.5 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + 0.75 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + 0.75 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 0 \\ -2 \end{pmatrix}\end{aligned}$$

- Finally, remembering that our vectors are augmented with a bias.
- We can equate the last entry in $\tilde{w}$ as the hyperplane offset b and write the separating
- Hyperplane equation $\mathbf{y} = \mathbf{w}\mathbf{x} + b$

$$\mathbf{w} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \text{ and } b = -2$$

For non-linear dataset

- Our goal, again, is to discover a separating hyperplane that accurately discriminates the two classes.
- Of course, it is obvious that no such hyperplane exists in the input space
- Therefore, we must use a nonlinear SVM (that is, we need to convert data from one feature space to another)



$$\Phi_1 \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{cases} \begin{pmatrix} 4 - x_2 + |x_1 - x_2| \\ 4 - x_1 + |x_1 - x_2| \end{pmatrix} & \text{if } \sqrt{x_1^2 + x_2^2} > 2 \\ \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} & \text{otherwise} \end{cases}$$